

Phase variance as a seismic quality-control attribute

Akshika Rohatgi¹, Andrey Bakulin¹, and Sergey Fomel¹

¹ University of Texas at Austin, Bureau of Economic Geology

Abstract

Seismic wavefields recorded on land are strongly distorted by near-surface heterogeneity, which introduces trace-specific, frequency-dependent phase perturbations that persist even after advanced time processing. Conventional processing relies primarily on surface-consistent deconvolution, which targets long- to mid-wavelength phase variability under the highly simplified assumption of surface consistency, equalizing large-scale trends and taming variability through overdetermination. However, this approximation is inherently unable to correct localized, non-surface-consistent phase distortions, and its effectiveness further degrades when such effects dominate, as is often the case for point-receiver data.

A separate and equally important limitation is that conventional workflows provide no direct, quantitative measure of phase reliability. Phase quality is therefore assessed only indirectly, typically through amplitude behavior or visual inspection, leaving residual phase disorder largely undiagnosed.

We introduce phase variance as a seismic quality-control attribute by treating seismic phases as circular random variables and analyzing local trace ensembles using circular statistics. This data-driven measure quantifies localized phase dispersion without phase unwrapping, enabling analysis of local phase trends and fluctuations without global assumptions or wavelet models. Phase variance is computed automatically and provides frequency-by-frequency classification of the data, ranging from coherent signal behavior to fully randomized, noise-dominated phase. Synthetic tests confirm that phase variance reliably captures imposed phase perturbations and their frequency dependence.

Application of phase variance analysis to field prestack land data shows that conventional processing reduces phase variability primarily in the low-to-intermediate frequency range and struggles within the noise cone, while the highest and lowest frequencies often show little improvement in phase coherence. Phase variance operates automatically over the full prestack volume, from shallow to deep, and frequency by frequency, providing a consistent, human-independent metric for defining effective bandwidth based on phase coherence and supporting phase-sensitive workflows such as AVO, migration, and full-waveform inversion.

Introduction

Seismic wavefields recorded at the surface are inevitably distorted by near-surface heterogeneity and small-scale elastic contrasts. These perturbations scatter energy and introduce complex, frequency-dependent distortions of amplitude and phase (Sato, Fehler, and Maeda, 2012). In land seismic processing, such effects have traditionally been mitigated through the extensive use of source and receiver arrays during acquisition (Meunier, 2011), followed by surface-consistent processing methods. Surface-consistent deconvolution and residual statics model the data using operators assigned separately to sources and receivers, implicitly assuming that near-surface distortions repeat in a predictable manner across traces (Taner, Koehler, and Alhilali, 1974; Taner and Koehler, 1981). Randomness is accommodated only in a limited sense, typically through trace-dependent static time shifts parameterized by one operator per source and one per receiver. These methods were deliberately designed to limit the number of degrees of freedom and overdetermine the estimation problem, enabling robust first-order corrections for statics and coupling, but without being grounded in a physical model of wave propagation through heterogeneous media.

Cary and Nagarajappa (2014a, 2014b) explicitly demonstrated that surface-consistent deconvolution can leave substantial residual phase errors and proposed additional surface-consistent phase corrections coupled with residual statics estimation. However, the underlying assumption remained that phase corrections are frequency-independent and surface-consistent. As a result, these approaches remain fundamentally unable to address the trace-to-trace, frequency-dependent phase variability that is abundant in modern single-sensor data. Several extensions have sought to relax strict surface consistency by introducing non-surface-consistent, time-varying statics that allow trace-dependent perturbations (Reilly et al., 2010). While these methods come closest to acknowledging the increased complexity of near-surface effects within conventional processing, they remain fundamentally constrained. By construction, they address only travel-time corrections and do not account for localized, frequency-dependent phase perturbations arising from small-scale scattering. Consequently, the available degrees of freedom remain insufficient to represent waveform distortions that vary continuously along the trace and differ for each source–receiver pair.

In practice, wave propagation through heterogeneous near-surface media is not surface consistent. Small-scale heterogeneity induces forward scattering and interference that generate non-surface-consistent, trace-specific phase perturbations (Bakulin et al., 2022a). These perturbations are not purely kinematic and cannot be corrected by time shifts alone. Instead, they modify the phase locally in time and frequency, even after advanced statics and deconvolution have been applied. As a result, residual phase disorder persists throughout prestack gathers, revealing a fundamental gap between classical processing assumptions and the physics of wave propagation in heterogeneous media (Stork et al., 2020; Bakulin et al., 2020).

Ensemble-based processing methods implicitly acknowledge the importance of phase coherence by exploiting redundancy in multi-channel seismic data. Historically, this redundancy was realized directly in acquisition through the use of source and receiver arrays, where local stacking in the time domain improved phase coherence but simultaneously conditioned phase and amplitude, a limitation perceived as detrimental to amplitude fidelity and high-frequency preservation (Newman and Mahoney, 1973). In seismology, phase-weighted stacks were introduced to explicitly prioritize phase coherence for signal detection, reflecting the recognized outsized importance of recovering correct phase from multi-channel recordings (Schimmel and Paulssen, 1997; Schimmel and Gallart, 2007). Phase-estimation and enhancement methods based on local attributes and blind deconvolution were later proposed to stabilize phase behavior (van der Baan, 2008; van der Baan and Fomel, 2009; Fomel and van der Baan, 2014; Holt and Lubrano, 2020). More recently, with the widespread adoption of point-receiver acquisition, ensemble-based phase conditioning has been reintroduced into reflection seismic processing through seismic time-frequency masking, where phase manipulation is decoupled from amplitude effects (Bakulin et al., 2023). While these methods effectively reduce incoherent phase locally, they treat phase variability primarily as a nuisance to be suppressed and do not explicitly analyze phase as a statistical quantity or quantify the underlying phase distributions that govern coherence loss.

Despite decades of advances in noise attenuation and imaging, and long-standing recognition of the critical role of phase coherence, seismic processing still lacks a routine, quantitative, and objective measure of phase reliability. In practice, phase quality is assessed indirectly through visual inspection, stack response, or amplitude-based attributes, relying on ad-hoc heuristics and human judgment rather than a standardized metric that isolates phase behavior or captures its frequency-dependent variability. This limitation is most evident in land environments with strong near-surface heterogeneity, where pronounced phase variability persists despite careful conventional processing (Bakulin et al., 2020; Stork, 2020). The problem becomes acute in desert environments such as the Middle East, where near-surface heterogeneity is particularly strong (Bakulin et al., 2022a; 2024), and reaches its extreme in highly scattering media such as basalts and crystalline rocks, where seismic imaging is often restricted to very low frequencies (Ziolkowski et al., 2003).

Recent work has reframed near-surface scattering as a multiplicative process that randomizes seismic phase, giving rise to speckle-like behavior in seismic wavefields (Bakulin et al., 2022a; 2023; Rohatgi et al., 2025). Analogous phenomena are well documented in optics and acoustics, where volumetric scatterers produce frequency-dependent phase fluctuations and coherence loss that are best described statistically (Goodman, 2007; Abbott and Thurstone, 1979). In seismic data, forward scattering on meter-scale heterogeneity generates multiple near-ballistic arrivals whose interference leads to rapid, trace-to-trace phase variations. These phase perturbations often dominate waveform degradation and increase with frequency, driving the wavefield toward a speckle regime in which individual traces appear chaotic, yet ensemble statistics remain stable and physically meaningful.

Modern land seismic acquisition is characterized by dense spatial sampling with point receivers rather than source-receiver arrays. While this increases spatial resolution and preserves high-frequency information, it also makes small-scale near-surface heterogeneity and trace-to-trace phase variability far more apparent within localized, densely sampled ensembles. In such settings, propagation through heterogeneous near-surface media renders the observed seismic phase effectively random from trace to trace.

This study adopts an explicitly stochastic viewpoint. Although the seismic source is deterministic, the resulting wavefield observed in prestack data reflects the cumulative effects of near-surface scattering and interference. Importantly, seismic phase is inherently circular rather than linear and must be treated accordingly. We introduce an ensemble-based framework that models seismic phase as a circular random variable and applies circular statistics to quantify phase variability within prestack gathers. We define a phase-variance attribute that summarizes frequency-dependent phase disorder. By shifting from single-trace phase estimates to ensemble phase distributions, the proposed framework provides a physically grounded measure of scattering-induced phase noise and establishes a transparent link among near-surface heterogeneity, phase disorder, and seismic coherence.

PHASE BEHAVIOR AND STATISTICS IN SEISMIC ANALYSIS

To motivate a statistical description of seismic phase, we start with controlled examples that show how phase coherence changes within local trace ensembles. Rather than interpreting phase trace by trace, we treat the phases at each frequency as a sample drawn from an underlying distribution and track how that distribution evolves as noise increases. We therefore analyze a sequence of cases spanning the full range of signal-to-noise ratios (SNR), anchored by two end members, a pure signal reference and a pure noise reference (Figure 1). These idealized limits provide intuition for the behavior encountered in real prestack gathers and establish a baseline for interpreting phase variability in more complex settings. Figure 1 organizes the sequence from fully coherent to fully randomized phase behavior. In the noise-free limit ($\text{SNR} \rightarrow \infty$), phases are aligned across traces, and the phase distribution is maximally concentrated. As noise is introduced, phase alignment degrades and the distribution broadens. In the opposite limit ($\text{SNR} \rightarrow -\infty$), the phases are uniformly distributed on $[-\pi, \pi]$, indicating complete loss of coherence. The following subsections describe these three regimes in detail and use them to motivate circular statistical measures of phase direction and dispersion.

```
1 using CSV, DataFrames, FFTW, Random, Plots
```

```
Precompiling Plots...
 1721.2 ms  ✓ libass_jll
 1493.9 ms  ✓ FFMPEG_jll
  978.8 ms  ✓ FFMPEG
 2134.8 ms  ✓ GR_jll
 2713.6 ms  ✓ GR
30802.3 ms  ✓ Plots
6 dependencies successfully precompiled in 41 seconds. 174 already precompiled.
```

```
T = 0.002
```

```
1 T = 0.002
```

```
fs = 500.0
```

```
1 fs = 1/T
```

```
num_wiggles = 10000
```

```
1 num_wiggles=10000
```

```

173x10000 Matrix{Float64}:
 0.000559426  0.000559426  0.000559426  ...  0.000559426  0.000559426
 0.000250176  0.000250176  0.000250176  ...  0.000250176  0.000250176
 1.46e-5      1.46e-5      1.46e-5      ...  1.46e-5      1.46e-5
-2.32e-5      -2.32e-5      -2.32e-5      ...  -2.32e-5      -2.32e-5
 5.82e-5      5.82e-5      5.82e-5      ...  5.82e-5      5.82e-5
 0.000118091  0.000118091  0.000118091  ...  0.000118091  0.000118091
 9.89e-5      9.89e-5      9.89e-5      ...  9.89e-5      9.89e-5
  ⋮
-6.97e-5      -6.97e-5      -6.97e-5      ...  -6.97e-5      -6.97e-5
-6.43e-6      -6.43e-6      -6.43e-6      ...  -6.43e-6      -6.43e-6
-0.000758544  -0.000758544  -0.000758544  ...  -0.000758544  -0.000758544
-0.00192232   -0.00192232   -0.00192232   ...  -0.00192232   -0.00192232
-0.0025316    -0.0025316    -0.0025316    ...  -0.0025316    -0.0025316
-0.00194126   -0.00194126   -0.00194126   ...  -0.00194126   -0.00194126

```

```

1 begin
2     using HTTP
3     file_id = "1suyqk05jKfG3As5iub7iuWMDFXibwLH4"
4     url = "https://drive.google.com/uc?export=download&id=$file_id"
5     response = HTTP.get(url)
6     klauder = DataFrame(CSV.File(IOBuffer(response.body))).signal[500:672]
7     clean_traces = repeat(klauder, 1, num_wiggles)
8 end

```

```
t = 0.0:0.0020116279069767443:0.34600000000000003
```

```
1 t = range(start=0, stop=length(klauder)*T, length=length(klauder))
```

```
plot_wiggles (generic function with 1 method)
```

```

1 function plot_wiggles(traces, t; step=200, scale=200, lw=1, color=:black, title="")
2     p = plot(size=(800, 600), title=title, ylabel="Time (s)", xlabel="Trace",
3             yflip=true, grid=false, legend=false)
4     for i in 1:step:size(traces, 2)
5         trc = traces[:, i]
6         plot!(p, trc .* scale .+ i, t, linecolor=color, lw=lw)
7     end
8     return p
9 end

```

trace_spectrum (generic function with 1 method)

```
1 function trace_spectrum(data, dt; extras=false)
2     nt, ntr = size(data)
3     freq = fftfreq(nt, 1/dt)
4     pos_inds = findall(x -> x > 0, freq) # excludes DC
5     nfreq = length(pos_inds)
6     phase_vals = zeros(Float32, nfreq, ntr)
7     amplitude_vals = zeros(Float32, nfreq, ntr)
8     f_zero = zeros(ComplexF64, 1, ntr)
9     for k in 1:ntr
10        fft_trace = fft(data[:, k])
11        f_zero[1, k] = fft_trace[1]
12        phase_vals[:, k] = angle.(fft_trace[pos_inds])
13        amplitude_vals[:, k] = abs.(fft_trace[pos_inds])
14    end
15    return extras ? (phase_vals, amplitude_vals, freq[pos_inds], f_zero) :
16        (phase_vals, amplitude_vals)
17 end
```

```
▶ (86×10000 Matrix{Float32}:
 -2.99687 -2.99687 -2.99687 -2.99687 ... -2.99687 -2.99687 -2.99687 2.25863, 86×10000
1 phase_clean, amplitude_clean, positive_freq, f_zero =
2 trace_spectrum(clean_traces, T, extras=true)
```

```
1 using CircStats
```

```
1 using StatsBase
```

circle (generic function with 1 method)

```
1 function circle(R; n=200)
2     θ = range(0, 2π, length=n)
3     return cos.(θ) .* R, sin.(θ) .* R
4 end
```

sector (generic function with 1 method)

```
1 function sector(θ1, θ2, r; n=50)
2     θ = range(θ1, θ2, length=n)
3     x = vcat(0.0, r .* cos.(θ), 0.0)
4     y = vcat(0.0, r .* sin.(θ), 0.0)
5     return Shape(x, y)
6 end
```

circ_hist_with_vm (generic function with 3 methods)

```
1 function circ_hist_with_vm(phases, binwidth,  $\mu$ =nothing,  $\kappa$ =nothing;
2     title_str="",
3     fillcol="#6fa76f",
4     linecol=:red,
5     lw=3.5,
6     show_vm=true
7 )
8     n = max(1, round(Int, 2 $\pi$ /binwidth))
9     binwidth = 2 $\pi$ /n
10    bins = range(0, stop=2 $\pi$ , length=n+1)
11    data = mod.(phases, 2 $\pi$ )
12    data .+= 10*eps(float(eltype(phases)))
13    hist = fit(Histogram, data, bins, closed=:left)
14    edges = hist.edges[1]
15    R = maximum(hist.weights)
16
17    p = plot(aspect_ratio=:equal, xlim=(-R, R), ylim=(-R, R), legend=false,
18    grid=false, title=title_str)
19    plot!(p, circle(R); lc=:black, fill=nothing)
20    plot!(p, sector.(edges[1:end-1], edges[2:end], hist.weights), fill=fillcol, lw=1,
21    showaxis=false)
22
23    # only draw von Mises curve if requested and parameters are provided
24    if show_vm && !isnothing( $\mu$ ) && !isnothing( $\kappa$ )
25         $\theta$  = range(0, 2 $\pi$ , length=2000)
26        r = @. 1 / (2 $\pi$  * besseli(0,  $\kappa$ )) * exp( $\kappa$  * cos( $\theta$  -  $\mu$ ))
27        rmax = maximum(r)
28        r = rmax == 0 ? r : (R * r / rmax)
29        x = r .* cos.( $\theta$ )
30        y = r .* sin.( $\theta$ )
31        plot!(p, x, y, color=linecol, lw=lw)
32    end
33    return p
34 end
```

add_noise (generic function with 1 method)

```
1 function add_noise(signal, num_wiggles, desired_SNR_dB)
2     noise_power = var(signal) / 10^(desired_SNR_dB / 10)
3     noisy_traces = zeros(Float64, length(signal), num_wiggles)
4     for i in 1:num_wiggles
5         noisy_traces[:,i] = signal .+ sqrt(noise_power) .* randn(size(signal))
6     end
7     return noisy_traces
8 end
```

```
1 noise_only = add_noise(klauder, num_wiggles, -100);
```

```
► (86×10000 Matrix{Float32}:  
  0.0790339 -1.18699 -1.5042 -1.83816 ... 1.60826 -2.06585 -1.45154 , 86×10000 Matrix{Float32})
```

```
1 phase_noise_only, amplitude_noise_only, positive_freq_noise_only, f_zero_noise_only =  
2 trace_spectrum(noise_only, T, extras=true)
```

```
1 noise_only_plot = noise_only*0.00002;
```

```
1 add_noise_traces5dB = add_noise(klauder, num_wiggles, -5);
```

```
► (86×10000 Matrix{Float32}:  
 -0.829526 2.82644 -2.68298 3.01479 ... 2.64224 3.03025 2.04705 2.2135 , 86×10000 Matrix{Float32})
```

```
1 phase_add_noise, amplitude_add_noise, positive_freq_add_noise, f_zero_add_noise =  
2 trace_spectrum(add_noise_traces5dB, T, extras=true)
```

```
1 using SpecialFunctions
```

kappa_from_R (generic function with 1 method)

```
1 function kappa_from_R(R)  
2     if R < 0.53  
3         return 2R + R^3 + (5R^5)/6  
4     elseif R < 0.85  
5         return -0.4 + 1.39R + 0.43/(1 - R)  
6     else  
7         return 1/(3R - 4R^2 + R^3)  
8     end  
9 end
```

```
1 using Distributions
```

multiplicative_noise (generic function with 1 method)

```
1 function multiplicative_noise(clean_traces, phase_clean, amp_clean, f_zero,
   positive_freq, fs; tau=0.004)
2     # — compute frequency-dependent kappa from von Mises model
3     sig = 2π .* positive_freq .* tau
4     num_samples = length(sig)
5     kappa_in = zeros(length(sig))
6     for (i, M) in enumerate(sig)
7         phi = randn(num_samples) .* M
8         z = exp.(1.0im .* phi)
9         phase = angle.(z)
10        r = circ_r(phase)
11        kappa_in[i] = circ_kappa(r)
12    end
13    # trim to match phase_clean rows
14    nf = size(phase_clean, 1)
15    kappa_in = kappa_in[1:nf]
16    # — apply multiplicative phase perturbations
17    perturbed_phases = similar(phase_clean)
18    for i in 1:size(phase_clean, 1)
19        pp = phase_clean[i, :]
20        kappat = kappa_in[i]
21        von_mises_dist = VonMises(0.0, kappat)
22        random = rand(von_mises_dist, size(phase_clean, 2))
23        perturbed_phases[i, :] .= pp .+ random
24    end
25    # — reconstruct perturbed signal via inverse FFT
26    new_f = amp_clean .* exp.(1.0im .* perturbed_phases)
27    perturbed_signal_half = zeros(ComplexF64, size(new_f, 1), size(new_f, 2))
28    for i in 1:size(new_f, 2)
29        perturbed_signal_half[:, i] = reverse(conj(new_f[:, i]))
30    end
31    perturbed_signal = vcat(f_zero, new_f, perturbed_signal_half)
32    ifft_perturbed_signal = zeros(Float64, size(perturbed_signal, 1),
size(perturbed_signal, 2))
33    for i in 1:size(perturbed_signal, 2)
34        ifft_perturbed_signal[:, i] = real(ifft(perturbed_signal[:, i]))
35    end
36    return ifft_perturbed_signal
37 end
```

```

mul_noise_traces =
173x10000 Matrix{Float64}:
 0.156983 -0.0641441 -0.160921 ... 0.0825942 0.193802 0.0865005
 0.154552 0.0647897 -0.107041 -0.00598182 0.273363 0.129827
 0.0483113 0.134202 0.00742442 -0.0608118 0.248904 0.0538134
-0.0982672 0.0753755 0.0649568 -0.0611109 0.169764 -0.111419
-0.286854 -0.039852 0.0698357 -0.0270499 0.0167454 -0.16604
-0.369024 -0.153458 -0.0136848 ... -0.0516859 -0.105348 -0.208379
-0.313479 -0.201171 -0.0344119 -0.034924 -0.188161 -0.191748
 ⋮
-0.105149 0.111166 -0.0427937 -0.0776317 -0.016415 0.0817943
-0.104578 0.103382 0.0562307 -0.0981363 -0.0780858 0.0158881
-0.0786073 0.0710951 0.127734 -0.0130124 -0.171277 -0.0395178
-0.0242323 -0.0530066 0.0921147 ... 0.073384 -0.193772 -0.0563492
 0.0376462 -0.153092 -0.0268155 0.137587 -0.103148 -0.0315304
 0.130691 -0.15534 -0.0991688 0.14941 0.0382933 0.0483312

1 mul_noise_traces = multiplicative_noise(
2   clean_traces, phase_clean, amplitude_clean, f_zero, positive_freq, fs; tau=0.004)

▶ (86x10000 Matrix{Float32}:
 -2.95969 -2.95887 -3.0414 -3.08063 ... -3.02232 -2.9873 -3.05996 , 86x10
1 phase_mul_noise, amplitude_mul_noise, positive_freq_mul_noise, f_zero_mul_noise =
2   trace_spectrum(mul_noise_traces, T, extras=true)

```

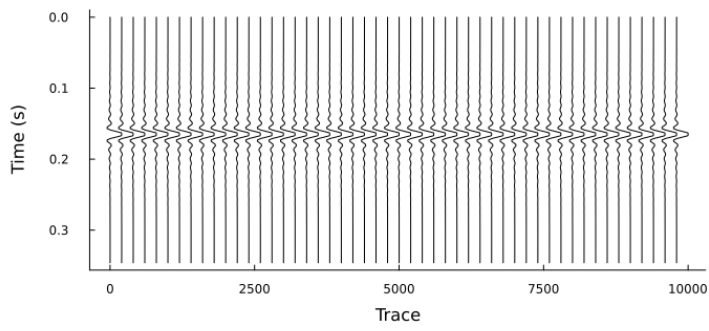
Signal only (SNR = ∞)

We begin with the noise-free reference case. Here the time-domain traces are perfectly repeatable within the local ensemble (Figure 1a), and the frequency-domain phases are coherently aligned across traces at all frequencies (Figure 1b). To describe this behavior statistically, we examine the distribution of phase values across the ensemble at each frequency, rather than individual phase spectra. For a multichannel dataset with N traces, define the set of phases at frequency ω as

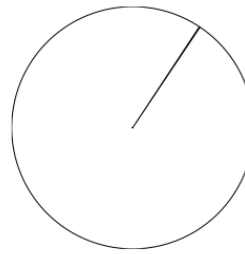
$$\Theta(\omega) = \{\theta_1(\omega), \theta_2(\omega), \dots, \theta_N(\omega)\} \quad (1)$$

In the absence of noise, all phases are identical, $\theta_1(\omega) = \theta_2(\omega) = \theta_N(\omega) = \theta_0(\omega)$, so the phase distribution collapses to a single value, which defines the lower bound of phase variability. This collapse is illustrated at 17 Hz and 52 Hz in Figures 1d and 1f, respectively, where phases are tightly concentrated, and the circular variance is zero ($V = 0.00$). This limiting case provides a baseline against which phase perturbations from additive noise and scattering-induced multiplicative distortions can be quantified.

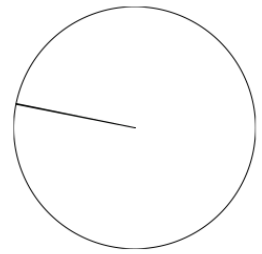
Clean traces (SNR = ∞)



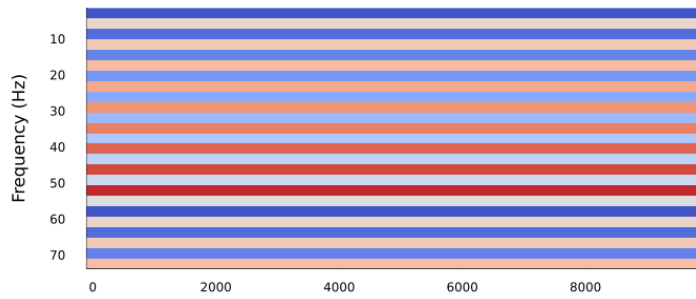
17 Hz



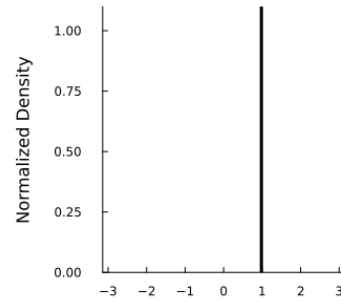
52 Hz



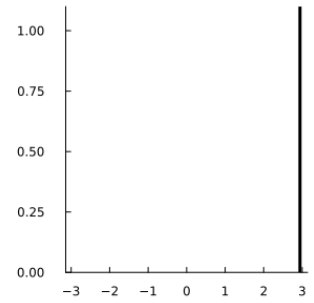
Clean phases



V = 0.0



V = 0.0



```

1 begin
2     V1d = round(max(0.0, 1 - circ_r(phase_clean[6, :])), digits=2)
3     V1f = round(max(0.0, 1 - circ_r(phase_clean[18, :])), digits=2)
4
5     p1a = plot_wiggles(clean_traces, t; title="Clean traces (SNR = ∞)")
6
7     p1b = heatmap(1:200:10000, positive_freq[1:25], phase_clean[1:25, 1:200:10000],
8                 yflip=true, c=:coolwarm, colorbar=false, clims=(-π, π),
9                 ylabel="Frequency (Hz)", title="Clean phases")
10
11     p1c = circ_hist_with_vm(phase_clean[6, :], 2π/1000; title_str="17 Hz",
12 show_vm=false)
13     p1e = circ_hist_with_vm(phase_clean[18, :], 2π/1000; title_str="52 Hz",
14 show_vm=false)
15
16     p1d = plot(xlims=(-π, π), ylims=(0, 1.1), title="V = $V1d",
17                 legend=false, grid=false, ylabel="Normalized Density")
18     vline!(p1d, [mean(phase_clean[6, :])], color=:black, lw=3)
19
20     p1f = plot(xlims=(-π, π), ylims=(0, 1.1), title="V = $V1f",
21                 legend=false, grid=false)
22     vline!(p1f, [mean(phase_clean[18, :])], color=:black, lw=3)
23
24     l = @layout [
25         [a{0.5h}; b{0.5h}] [c e; d f]
26     ]
27
28     plot(p1a, p1b, p1c, p1e, p1d, p1f,
29         layout=l,
30         size=(1400, 700),
31         left_margin=8Plots.mm,
32         bottom_margin=8Plots.mm,
33         top_margin=4Plots.mm)
34 end

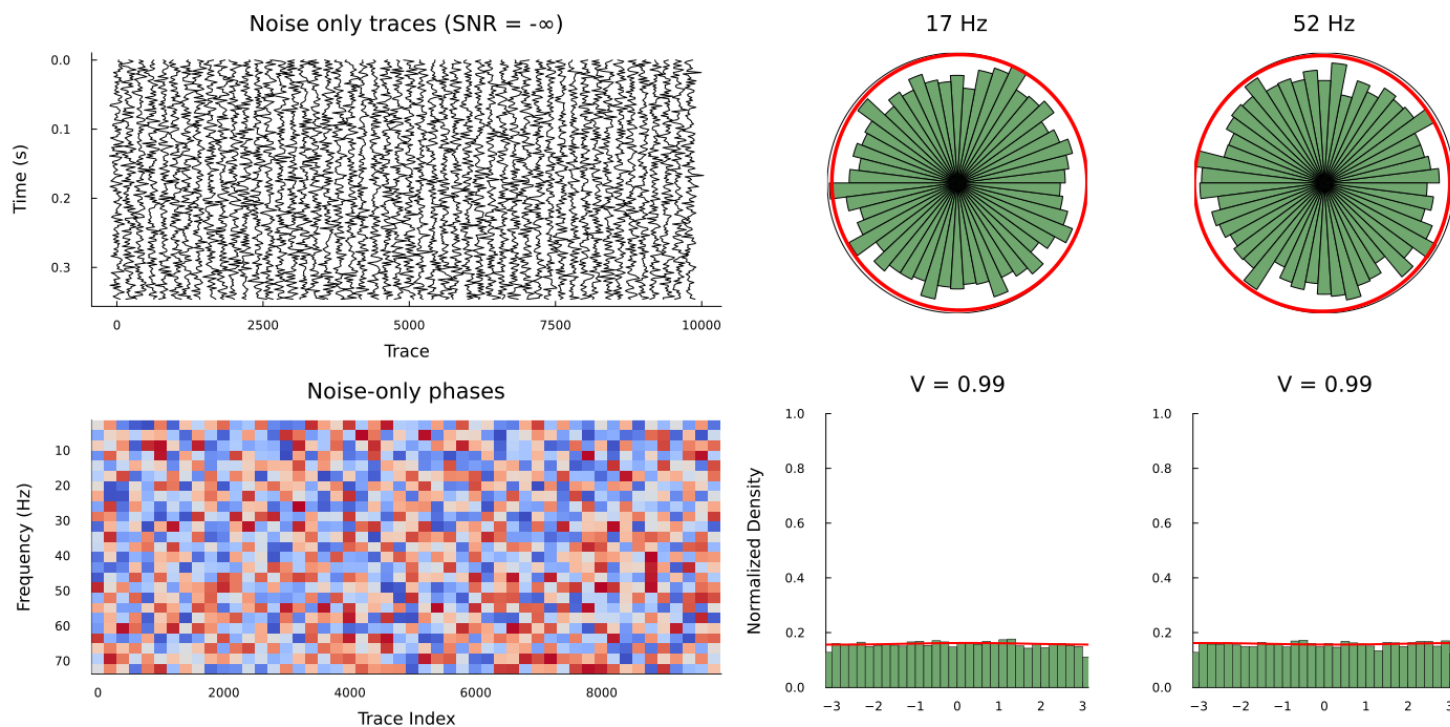
```

Noise only (SNR = $-\infty$)

At the opposite extreme, we consider the noise-only limit, which defines the upper bound of phase variability (Figure 1g). With no coherent signal present, the frequency-domain phases are random and show no alignment across traces (Figure 1h). For a local ensemble of N traces, the phase set at frequency ω is again given by equation (1), but the phases are now mutually uncorrelated across channels. As illustrated in Figures 1i and 1k, the phases form a uniform circular distribution over $[-\pi, \pi]$ at all frequencies,

$$\theta_k \sim U(-\pi, \pi), \quad k = 1, 2, \dots, N. \quad (2)$$

This case corresponds to a complete loss of phase coherence, with circular variance equal to one. Goodman (2007) showed that when a wavefield is modeled as a superposition of many independent scatterers with random phases, the resulting phase of the complex field is uniformly distributed on the circle. In the seismic context, Bakulin et al. (2022a) observed the same limiting behavior within a random multiplicative (speckle) framework, confirming that the noise-only end member exhibits uniform phase statistics.



```

1 begin
2     # circular variance values
3     V1j = round(max(0.0, 1 - circ_r(phase_noise_only[6, :])), digits=2)
4     V1l = round(max(0.0, 1 - circ_r(phase_noise_only[18, :])), digits=2)
5
6     # circular mean and kappa for noise only
7     mean_noise_only_17 = circ_mean(phase_noise_only[6, :]).μ
8     kappa_noise_only_17 = kappa_from_R(circ_r(phase_noise_only[6, :]))
9     mean_noise_only_52 = circ_mean(phase_noise_only[18, :]).μ
10    kappa_noise_only_52 = kappa_from_R(circ_r(phase_noise_only[18, :]))
11
12    θ_range_n = range(-π, π, length=500)
13
14    # 1g - wiggles
15    p1g = plot_wiggles(noise_only_plot, t; title="Noise only traces (SNR =  $-\infty$ )")
16
17    # 1h - phase heatmap
18    p1h = heatmap(1:200:10000, positive_freq_noise_only[1:25], phase_noise_only[1:25,
19    1:200:10000],
20                  yflip=true, c=:coolwarm, colorbar=false, clim=(-π, π),
21                  ylabel="Frequency (Hz)", title="Noise-only phases",
22                  xlabel="Trace Index")
23
24    # 1i - circular histogram 17 Hz

```

```

24 p1i = circ_hist_with_vm(phase_noise_only[6, :], 2π/50,
25                         mean_noise_only_17, kappa_noise_only_17;
26                         title_str="17 Hz", show_vm=true)
27
28 # 1k - circular histogram 52 Hz
29 p1k = circ_hist_with_vm(phase_noise_only[18, :], 2π/50,
30                         mean_noise_only_52, kappa_noise_only_52;
31                         title_str="52 Hz", show_vm=true)
32
33 # 1j - linear phase distribution 17 Hz
34 p1j = plot(xlims=(-π, π), ylims=(0, 1), title="V = $V1j",
35            legend=false, grid=false, ylabel="Normalized Density")
36 histogram!(p1j, phase_noise_only[6, :], normalized=:pdf, bins=50,
37            fillcolor="#6fa76f", linecolor=:black, lw=0.5)
38 pdf_vm_n1 = @. exp(kappa_noise_only_17 * cos(θ_range_n - mean_noise_only_17)) /
39              (2π * besseli(0, kappa_noise_only_17))
40 plot!(p1j, θ_range_n, pdf_vm_n1, color=:red, lw=2)
41
42 # 1l - linear phase distribution 52 Hz
43 p1l = plot(xlims=(-π, π), ylims=(0, 1), title="V = $V1l",
44            legend=false, grid=false)
45 histogram!(p1l, phase_noise_only[18, :], normalized=:pdf, bins=50,
46            fillcolor="#6fa76f", linecolor=:black, lw=0.5)
47 pdf_vm_n2 = @. exp(kappa_noise_only_52 * cos(θ_range_n - mean_noise_only_52)) /
48              (2π * besseli(0, kappa_noise_only_52))
49 plot!(p1l, θ_range_n, pdf_vm_n2, color=:red, lw=2)
50
51 # — layout
52 l_noise = @layout [
53     [a{0.5h}; b{0.5h}] [c e; d f]
54 ]
55
56 plot(p1g, p1h, p1i, p1k, p1j, p1l,
57      layout=l_noise,
58      size=(1400, 700),
59      left_margin=8Plots.mm,
60      bottom_margin=8Plots.mm,
61      top_margin=4Plots.mm)
62 end

```

⚠ Requirements for confidence levels not met.

⚠ Requirements for confidence levels not met.

Noisy signal ($-\infty < \text{SNR} < \infty$)

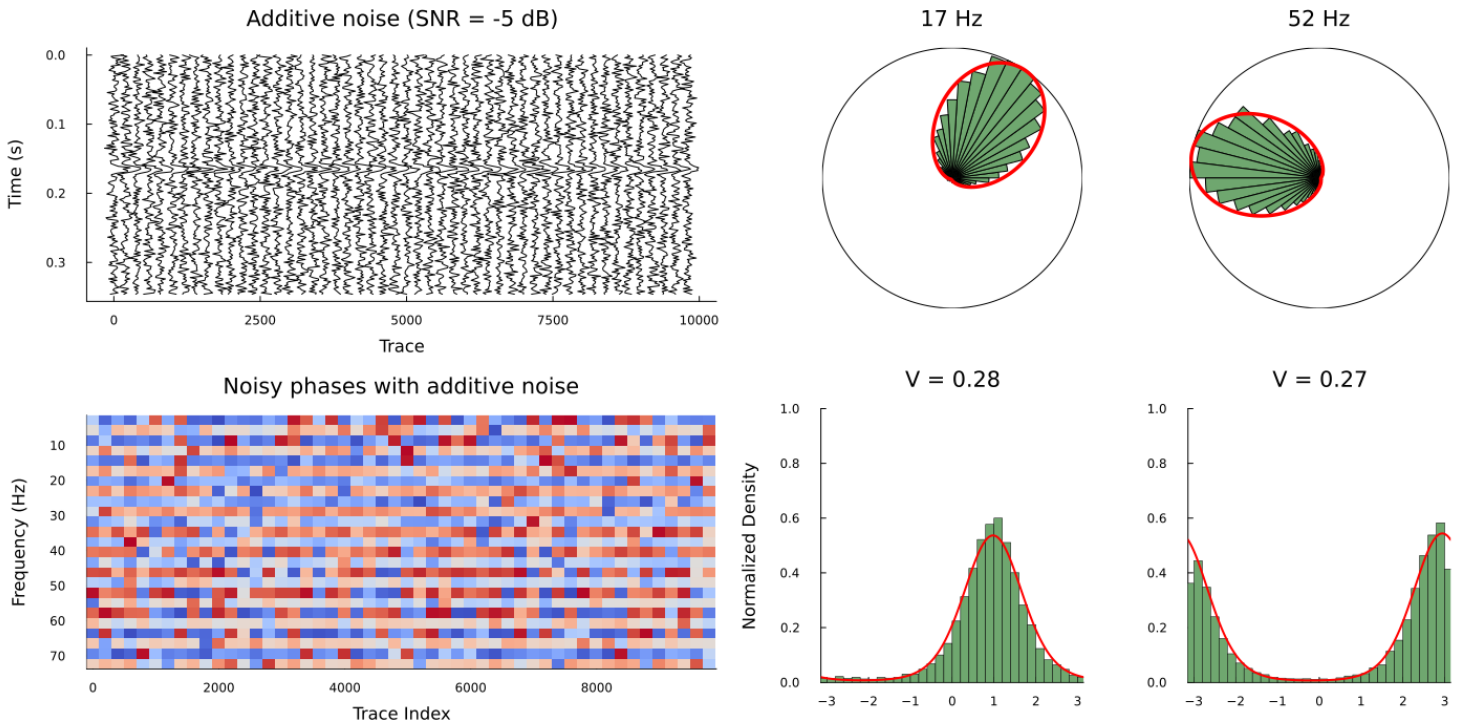
Most seismic data lie between the two end members and therefore contain both coherent reflections and noise. In this intermediate regime, reflection events are contaminated by either additive noise (Figure 1m) or multiplicative distortions associated with scattering (Figure 1s), so phase coherence is

partially preserved but systematically perturbed, leading to broadened phase distributions within local ensembles.

As a conceptual model, we write the phase of the k -th trace at frequency ω as

$$\theta_k(\omega) = \theta_0(\omega) + \delta_k(\omega), \quad k = 1, 2, \dots, N, \quad (3)$$

where $\theta_0(\omega)$ denotes the unperturbed signal phase and $\delta_k(\omega)$ is a symmetric random perturbation. As noise increases, the dispersion of $\{\theta_k(\omega)\}$ around $\theta_0(\omega)$ grows and the distribution broadens. To reproduce phase behavior observed in challenging land data, we adopt the multiplicative noise model of Bakulin et al. (2022a, 2023), which captures the key feature that scattering-induced phase perturbations increase with frequency; accordingly, phase distributions remain compact at low frequencies but broaden substantially at higher frequencies (Figures 1v and 1x). A practical complication is that seismic phase is wrapped to $[-\pi, \pi]$, which makes linear summaries and linear histograms misleading, for example, through apparent bimodality caused purely by wrapping (Figures 1r and 1x). Although phase unwrapping is common in radar, InSAR, and optical interferometry (Zebker, 1998), unwrapping becomes unstable in noisy or incoherent regions (Feigl et al., 2007) and is problematic for seismic traces (Shatilo, 1992). These issues motivate treating seismic phase as circular data and analyzing it with circular statistics (Mardia and Jupp, 2000; Mohammad et al., 2021; Rohatgi et al., 2025). Circular statistics operate directly on wrapped phases, avoid boundary artifacts at $-\pi$ and π , and provide robust measures of mean phase direction and dispersion, forming the basis for the phase-variance attributes introduced next (Bakulin et al., 2025; Rohatgi et al., 2025).



```

3  # circular variance values
4  V1p = round(max(0.0, 1 - circ_r(phase_add_noise[6, :])), digits=2)
5  V1r = round(max(0.0, 1 - circ_r(phase_add_noise[18, :])), digits=2)
6
7  # circular mean and kappa for additive noise
8  mean_add_noise_17 = circ_mean(phase_add_noise[6, :]).μ
9  kappa_add_noise_17 = kappa_from_R(circ_r(phase_add_noise[6, :]))
10 mean_add_noise_52 = circ_mean(phase_add_noise[18, :]).μ
11 kappa_add_noise_52 = kappa_from_R(circ_r(phase_add_noise[18, :]))
12
13 θ_range = range(-π, π, length=500)
14
15 # 1m - wiggles
16 p1m = plot_wiggles(add_noise_traces5dB, t; title="Additive noise (SNR = -5 dB)")
17
18 # 1n - phase heatmap
19 p1n = heatmap(1:200:10000, positive_freq_add_noise[1:25], phase_add_noise[1:25,
1:200:10000],
20               yflip=true, c=:coolwarm, colorbar=false, clim=(-π, π),
21               ylabel="Frequency (Hz)", title="Noisy phases with additive noise",
22               xlabel="Trace Index")
23
24 # 1o - circular histogram 17 Hz
25 p1o = circ_hist_with_vm(phase_add_noise[6, :], 2π/50,
26                          mean_add_noise_17, kappa_add_noise_17;
27                          title_str="17 Hz", show_vm=true)
28
29 # 1q - circular histogram 52 Hz
30 p1q = circ_hist_with_vm(phase_add_noise[18, :], 2π/50,
31                          mean_add_noise_52, kappa_add_noise_52;
32                          title_str="52 Hz", show_vm=true)
33
34 # 1p - linear phase distribution 17 Hz
35 p1p = plot(xlims=(-π, π), ylims=(0, 1), title="V = $V1p",
36            legend=false, grid=false, ylabel="Normalized Density")
37 histogram!(p1p, phase_add_noise[6, :], normalized=:pdf, bins=50,
38            fillcolor="#6fa76f", linecolor=:black, lw=0.5)
39 pdf_vm1 = @. exp(kappa_add_noise_17 * cos(θ_range - mean_add_noise_17)) /
40            (2π * besseli(0, kappa_add_noise_17))
41 plot!(p1p, θ_range, pdf_vm1, color=:red, lw=2)
42
43 # 1r - linear phase distribution 52 Hz
44 p1r = plot(xlims=(-π, π), ylims=(0, 1), title="V = $V1r",
45            legend=false, grid=false)
46 histogram!(p1r, phase_add_noise[18, :], normalized=:pdf, bins=50,
47            fillcolor="#6fa76f", linecolor=:black, lw=0.5)
48 pdf_vm2 = @. exp(kappa_add_noise_52 * cos(θ_range - mean_add_noise_52)) /
49            (2π * besseli(0, kappa_add_noise_52))
50 plot!(p1r, θ_range, pdf_vm2, color=:red, lw=2)
51
52 # — layout
53 l_add = @layout [
54     [a{0.5h}; b{0.5h}] [c e; d f]
55

```

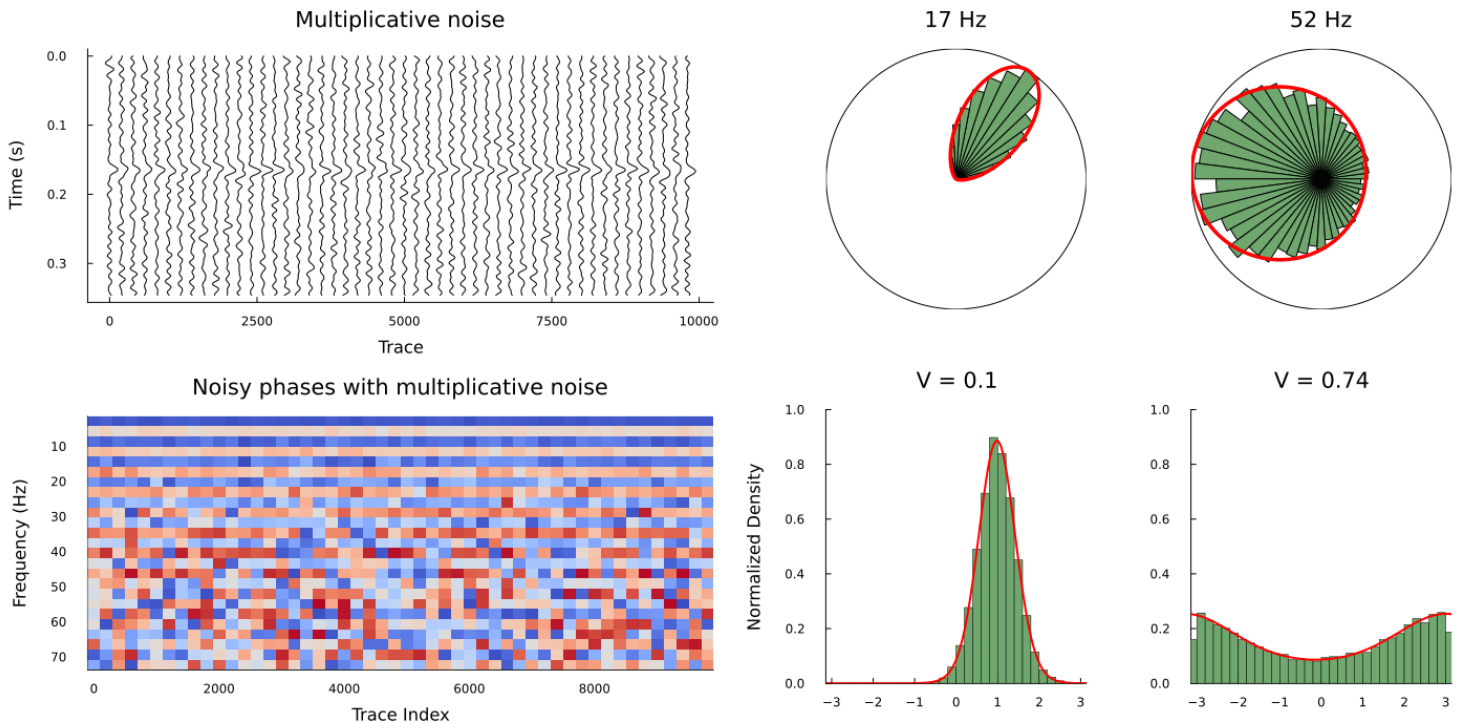


```

56 ]
57
58 plot(p1m, p1n, p1o, p1q, p1p, p1r,
59      layout=l_add,
60      size=(1400, 700),
61      left_margin=8Plots.mm,
62      bottom_margin=8Plots.mm,
        top_margin=4Plots.mm)

```

end



```

1 begin
2   # circular variance values
3   V1v = round(max(0.0, 1 - circ_r(phase_mul_noise[6, :])), digits=2)
4   V1x = round(max(0.0, 1 - circ_r(phase_mul_noise[18, :])), digits=2)
5
6   # circular mean and kappa for multiplicative noise
7   mean_mul_noise_17 = circ_mean(phase_mul_noise[6, :]).μ
8   kappa_mul_noise_17 = kappa_from_R(circ_r(phase_mul_noise[6, :]))
9   mean_mul_noise_52 = circ_mean(phase_mul_noise[18, :]).μ
10  kappa_mul_noise_52 = kappa_from_R(circ_r(phase_mul_noise[18, :]))
11
12  θ_range_m = range(-π, π, length=500)
13
14  # 1s - wiggles
15  p1s = plot_wiggles(mul_noise_traces, t; title="Multiplicative noise")
16
17  # 1t - phase heatmap
18  p1t = heatmap(1:200:10000, positive_freq_mul_noise[1:25], phase_mul_noise[1:25,
19  1:200:10000],
20               yflip=true, c=:coolwarm, colorbar=false, clims=(-π, π),
21               ylabel="Frequency (Hz)", title="Noisy phases with multiplicative

```

```

22
23 # 1u - circular histogram 17 Hz
24 p1u = circ_hist_with_vm(phase_mul_noise[6, :], 2π/50,
25                         mean_mul_noise_17, kappa_mul_noise_17;
26                         title_str="17 Hz", show_vm=true)
27
28 # 1w - circular histogram 52 Hz
29 p1w = circ_hist_with_vm(phase_mul_noise[18, :], 2π/50,
30                         mean_mul_noise_52, kappa_mul_noise_52;
31                         title_str="52 Hz", show_vm=true)
32
33 # 1v - linear phase distribution 17 Hz
34 p1v = plot(xlims=(-π, π), ylims=(0, 1), title="V = $V1v",
35           legend=false, grid=false, ylabel="Normalized Density")
36 histogram!(p1v, phase_mul_noise[6, :], normalized=:pdf, bins=50,
37           fillcolor="#6fa76f", linecolor=:black, lw=0.5)
38 pdf_vm_m1 = @. exp(kappa_mul_noise_17 * cos(θ_range_m - mean_mul_noise_17)) /
39           (2π * besseli(0, kappa_mul_noise_17))
40 plot!(p1v, θ_range_m, pdf_vm_m1, color=:red, lw=2)
41
42 # 1x - linear phase distribution 52 Hz
43 p1x = plot(xlims=(-π, π), ylims=(0, 1), title="V = $V1x",
44           legend=false, grid=false)
45 histogram!(p1x, phase_mul_noise[18, :], normalized=:pdf, bins=50,
46           fillcolor="#6fa76f", linecolor=:black, lw=0.5)
47 pdf_vm_m2 = @. exp(kappa_mul_noise_52 * cos(θ_range_m - mean_mul_noise_52)) /
48           (2π * besseli(0, kappa_mul_noise_52))
49 plot!(p1x, θ_range_m, pdf_vm_m2, color=:red, lw=2)
50
51 # — layout
52 l_mul = @layout [
53     [a{0.5h}; b{0.5h}] [c e; d f]
54 ]
55
56 plot(p1s, p1t, p1u, p1w, p1v, p1x,
57     layout=l_mul,
58     size=(1400, 700),
59     left_margin=8Plots.mm,
60     bottom_margin=8Plots.mm,
61     top_margin=4Plots.mm)
62 end

```


CIRCULAR STATISTICS FOR SEISMIC PHASE ANALYSIS

Circular Mean : Measure of mean phase direction

For an ensemble of seismic traces, the complex spectra can be normalized to unit magnitude so that only phase information is retained, independent of amplitude. This normalization ensures that phase statistics are not biased by variations in signal strength across traces. Figure 2 illustrates this procedure: noisy complex spectra (Figure 2a) are represented in the complex plane and then projected onto the unit circle (Figure 2b), preserving phase while removing amplitude effects.

At a given frequency ω , the phase of each trace is represented as a unit phasor

$$e^{i\theta_1(\omega)}, e^{i\theta_2(\omega)}, \dots, e^{i\theta_N(\omega)}, \quad (4)$$

where for a complex signal $X_k(\omega)$, $\theta_k(\omega) = \arg[X_k(\omega)]$ is the phase of the k -th trace and N is the number of traces in the local ensemble.

The preferred phase direction is characterized by the circular mean $\bar{\theta}(\omega)$, defined as the direction of the resultant vector formed by summing the unit phasors. At a fixed frequency, the cosine and sine components are averaged as:

$$\bar{C}(\omega) = \frac{1}{N} \sum_{k=1}^N \cos \theta_k(\omega), \quad \bar{S}(\omega) = \frac{1}{N} \sum_{k=1}^N \sin \theta_k(\omega). \quad (5)$$

The length of the mean resultant vector is

$$\bar{R}(\omega) = \sqrt{\bar{C}^2(\omega) + \bar{S}^2(\omega)}, \quad (6)$$

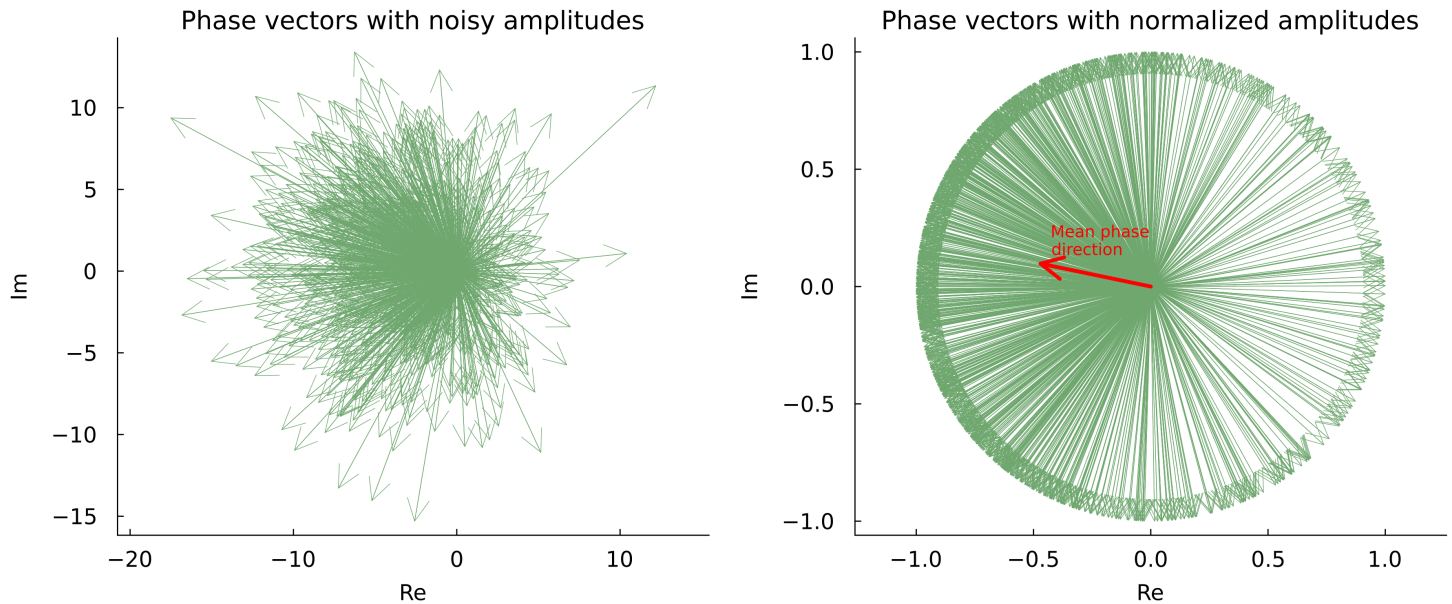
and the circular mean direction is given by

$$\bar{\theta}(\omega) = \text{atan2}(\bar{S}(\omega), \bar{C}(\omega)). \quad (7)$$

The mean direction is undefined when $\bar{R}(\omega) = 0$, corresponding to a uniform phase distribution with

no preferred direction.

It is important to emphasize that the circular mean $\bar{\theta}$ is not equivalent to the arithmetic average $(\theta_1 + \theta_2 + \dots + \theta_N)/N$. Instead, it represents the direction of the geometric vector sum of the unit phasors and correctly accounts for the periodic nature of phase. This distinction is critical when analyzing wrapped seismic phases, particularly in the presence of noise.



```
1 begin
2     # generate additive noise at -10 dB
3     add_noise_10dB = add_noise(klauder, num_wiggles, -10)
4     phase_add_noise_10dB, amplitude_add_noise_10dB, positive_freq_add_noise_10dB,
5     f_zero_add_noise_10dB =
6         trace_spectrum(add_noise_10dB, T, extras=true)
7
8     freq_idx = 18 # 52 Hz
9
10    signal_vector_10 = amplitude_add_noise_10dB[freq_idx, :] .* exp.(1.0im .*
11    phase_add_noise_10dB[freq_idx, :])
12    phase_vector_10 = exp.(1.0im .* phase_add_noise_10dB[freq_idx, :])
13
14    xs_phase_10 = real.(phase_vector_10)
15    ys_phase_10 = imag.(phase_vector_10)
16    xs_signal_10 = real.(signal_vector_10)
17    ys_signal_10 = imag.(signal_vector_10)
18
19    step = 10
20    idx_10 = 1:step:length(xs_phase_10)
21
22    xs_phase_sub_10 = xs_phase_10[idx_10]
23    ys_phase_sub_10 = ys_phase_10[idx_10]
24    xs_signal_sub_10 = xs_signal_10[idx_10]
25    ys_signal_sub_10 = ys_signal_10[idx_10]
26
27    mean_phase_10 = circ_mean(phase_add_noise_10dB[freq_idx, :]).μ
```

```

27     mean_r_10      = circ_r(phase_add_noise_10dB[freq_idx, :])
28
29     # 2a - noisy complex spectra
30     p2a = quiver(zeros(length(xs_signal_sub_10)), zeros(length(ys_signal_sub_10)));
31             quiver=(xs_signal_sub_10, ys_signal_sub_10),
32             aspect_ratio=1,
33             xlabel="Re", ylabel="Im",
34             title="Phase vectors with noisy amplitudes",
35             legend=false, linecolor="#6fa76f", arrow=true, lw=0.5, grid=false)
36
37     # 2b - unit phasors
38     p2b = quiver(zeros(length(xs_phase_sub_10)), zeros(length(ys_phase_sub_10)));
39             quiver=(xs_phase_sub_10, ys_phase_sub_10),
40             aspect_ratio=1,
41             xlabel="Re", ylabel="Im",
42             title="Phase vectors with normalized amplitudes",
43             legend=false, linecolor="#6fa76f", arrow=true, lw=0.5, grid=false)
44     quiver!(p2b, [0.0], [0.0];
45             quiver=([mean_r_10 * cos(mean_phase_10)], [mean_r_10 *
46 sin(mean_phase_10)]),
47             linecolor=:red, arrow=true, lw=3)
48     annotate!(p2b, mean_r_10 * cos(mean_phase_10) + 0.05, mean_r_10 *
49 sin(mean_phase_10) + 0.1,
50             text("Mean phase\ndirection", :red, 9, :left))
51
52     plot(p2a, p2b,
53         layout=(1, 2),
54         size=(1200, 500),
55         fontsize=12, labelfontsize=12,
56         legendfontsize=12, tickfontsize=12,
57         left_margin=8Plots.mm,
58         bottom_margin=8Plots.mm,
59         dpi=300)
60
61 end

```

Circular Variance: Measure of phase variability

While the mean direction characterizes the central tendency of seismic phases within an ensemble, it is equally important to quantify how tightly the phases cluster around that direction. This variability is measured by the mean resultant length $\bar{R}$ (equation 6), which takes values between 0 and 1. Values of $\bar{R}$ close to 1 indicate strong phase coherence, whereas values near 0 indicate highly dispersed, incoherent phases,

$$0 \leq \bar{R}(\omega) \leq 1. \quad (8)$$

Although the mean resultant length is the fundamental quantity in circular statistics, it is often more

intuitive, particularly by analogy with linear statistics, to express phase variability in terms of circular variance, defined as

$$V(\omega) = 1 - \bar{R}(\omega), \quad (9)$$

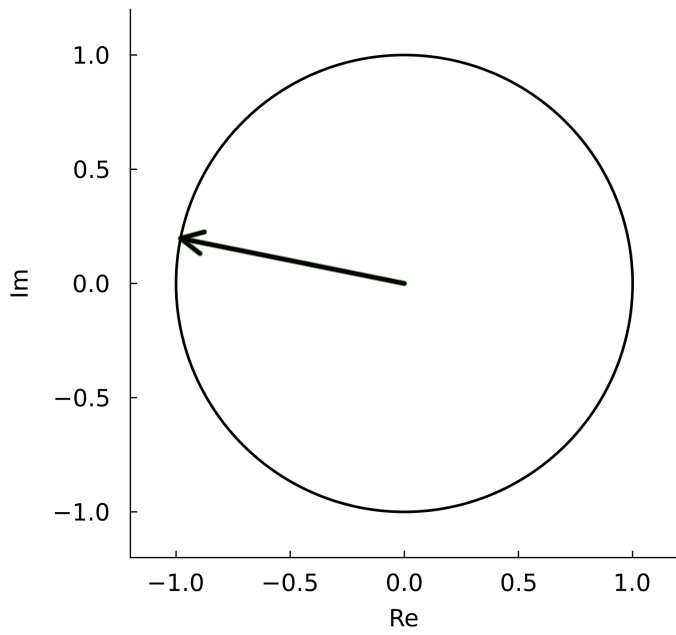
where $0 \leq V(\omega) \leq 1$.

In the ideal signal-only case, all phase vectors are aligned and point in the same direction, yielding $\bar{R}(\omega) = 1$ and $V(\omega) = 0$, corresponding to perfect phase coherence (Figure 3a). In contrast, when the data consist solely of noise, phase vectors are uniformly distributed around the unit circle, resulting in $\bar{R}(\omega) \approx 0$ and $V(\omega) \approx 1$, indicating maximal phase variability (Figure 3b). Most realistic seismic situations lie between these two extremes. For example, with moderate additive noise at -5 dB, the phase vectors retain a preferred direction but exhibit noticeable dispersion, producing $\bar{R}(\omega) \approx 0.73$ and $V(\omega) \approx 0.27$ (Figure 3c). Under stronger or frequency-dependent phase perturbations, such as those associated with multiplicative noise, the phase vectors spread further and the circular variance increases accordingly, reaching values around $V(\omega) \approx 0.5$ (Figure 3d). Although circular statistics provide a general framework for analyzing wrapped seismic phases, no specific parametric distribution is required to compute the mean direction or circular variance.

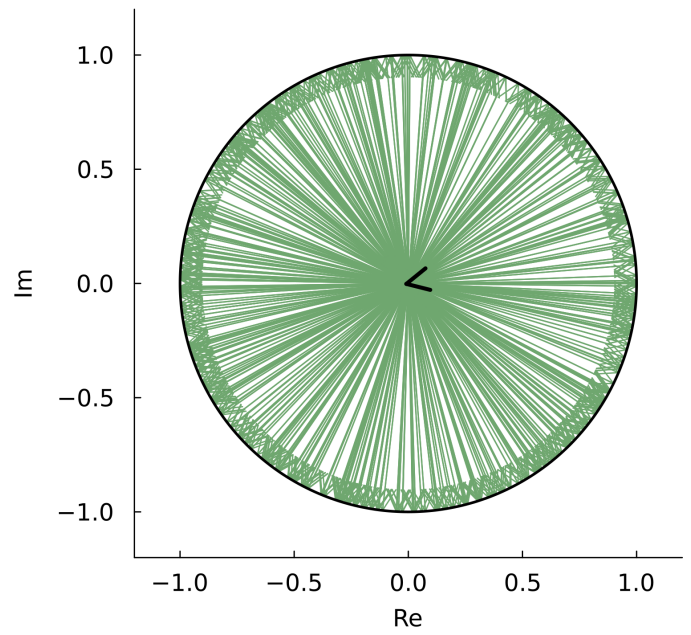
In this study, the von Mises distribution is introduced as a convenient conceptual model, as it is the circular analogue of the normal distribution, providing an intuitive interpretation of phase clustering, dispersion, and a well-defined center of gravity when a single dominant signal event is present within the analysis window.

However, our analysis does not depend on the von Mises assumption. Instead, we primarily use the circular variance $V(\omega)$, which is defined directly from the mean resultant length and can be estimated nonparametrically from phase ensembles. Circular variance is a robust, model-free observable that remains stable under strong noise contamination, avoids phase-unwrapping ambiguities, and can be computed consistently across frequencies, offsets, and processing stages, making it well suited as a practical QC and diagnostic metric for seismic phase behavior.

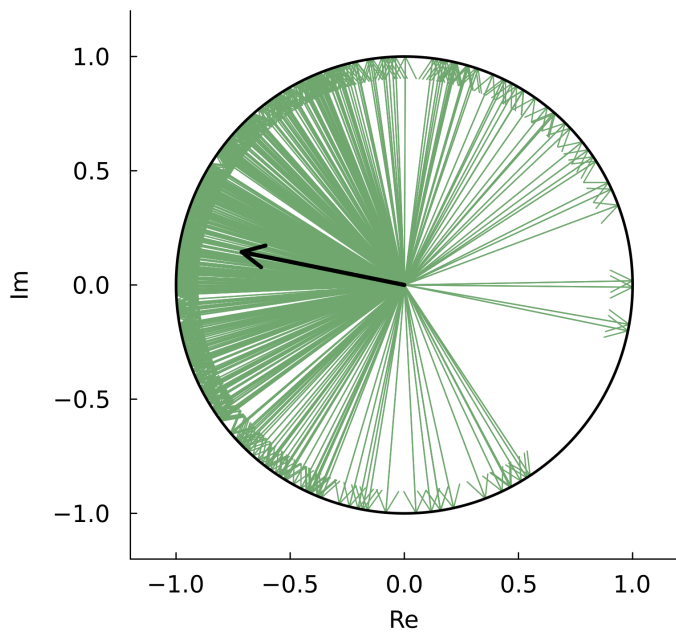
Signal Only



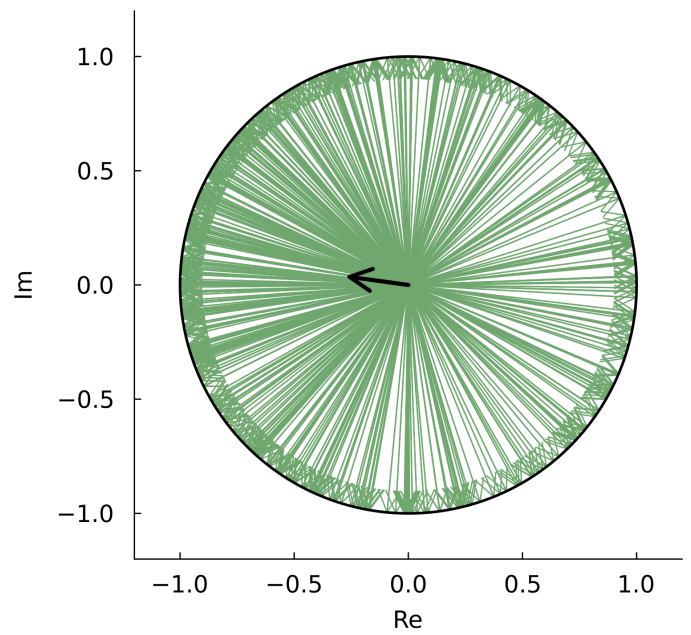
Noise Only



Additive Noise



Multiplicative Noise



```

1 begin
2      $\theta_{\text{circ\_fig3}}$  = range(- $\pi$ ,  $\pi$ ; length=500)
3      $\text{cx\_fig3}$ ,  $\text{cy\_fig3}$  = cos.( $\theta_{\text{circ\_fig3}}$ ), sin.( $\theta_{\text{circ\_fig3}}$ )
4
5     function make_phasor(phases, ttl; step=20, lw=1)
6         pv = exp.(1.0im .* phases)
7         x = real.(pv)[1:step:end]
8         y = imag.(pv)[1:step:end]
9         p = quiver(zeros(length(x)), zeros(length(y));
10                    quiver=(x, y),
11                    aspect_ratio=1,
12                    xlabel="Re", ylabel="Im",
13                    legend=false, linecolor="#6fa76f",
14                    arrow=true, lw=lw, grid=false,
15                    title=ttl)
16         plot!(p,  $\text{cx\_fig3}$ ,  $\text{cy\_fig3}$ , color=:black, lw=2)
17         # resultant vector R
18         R = circ_r(phases)
19          $\theta_m$  = circ_mean(phases). $\mu$ 
20         quiver!(p, [0.0], [0.0];
21                 quiver=([R * cos( $\theta_m$ )], [R * sin( $\theta_m$ )]),
22                 linecolor=:black, arrow=true, lw=3)
23         xlims!(p, -1.2, 1.2); ylims!(p, -1.2, 1.2)
24         return p
25     end
26
27     p3a = make_phasor(phase_clean[18, :], "Signal Only"; step=80, lw=4)
28     p3b = make_phasor(phase_noise_only[18, :], "Noise Only"; step=20, lw=1)
29     p3c = make_phasor(phase_add_noise[18, :], "Additive Noise"; step=20, lw=1)
30     p3d = make_phasor(phase_mul_noise[18, :], "Multiplicative Noise"; step=20, lw=1)
31
32     plot(p3a, p3b, p3c, p3d,
33          layout=(2, 2),
34          size=(1100, 1100),
35          margin=5Plots.mm,
36          fontsize=12, labelfontsize=12,
37          legendfontsize=12, tickfontsize=12,
38          xguidefontsize=12, yguidefontsize=12,
39          dpi=300)
40 end

```

⚠ Requirements for confidence levels not met.

The von Mises Distribution

The von Mises distribution, introduced by von Mises (1918) as the circular analogue of the normal distribution, has been widely used to model directional or wrapped data. It has found broad application in acoustics, optical interferometry, and InSAR, where phase is inherently periodic and

must be analyzed without unwrapping (Feigl et al., 2009; Jiang et al., 2019; Takamichi et al., 2018; Nugraha et al., 2019). In these fields, von Mises models are commonly used to represent phase variability, reconstruct phase information, and analyze noisy or wrapped phase measurements in a probabilistically consistent manner. In seismology, Gosselin (2022) applied von Mises distributions to model uncertainty in circular phase spectra of surface-wave dispersion, while Rohatgi et al. (2025) used circular statistics to quantify phase coherence within local seismic ensembles.

This distribution is parameterized by the mean direction $\bar{\theta}$ and the concentration parameter κ . Its probability density function is given as:

$$f(\theta; \bar{\theta}, \kappa) = \frac{1}{2\pi I_0(\kappa)} e^{\kappa \cos(\theta - \bar{\theta})}, \quad (10)$$

where I_0 denotes the modified Bessel function of the first kind and order zero, defined as

$$I_0(\kappa) = \frac{1}{2\pi} \int_0^{2\pi} e^{\kappa \cos \theta} d\theta. \quad (11)$$

The function I_0 also admits the power series expansion

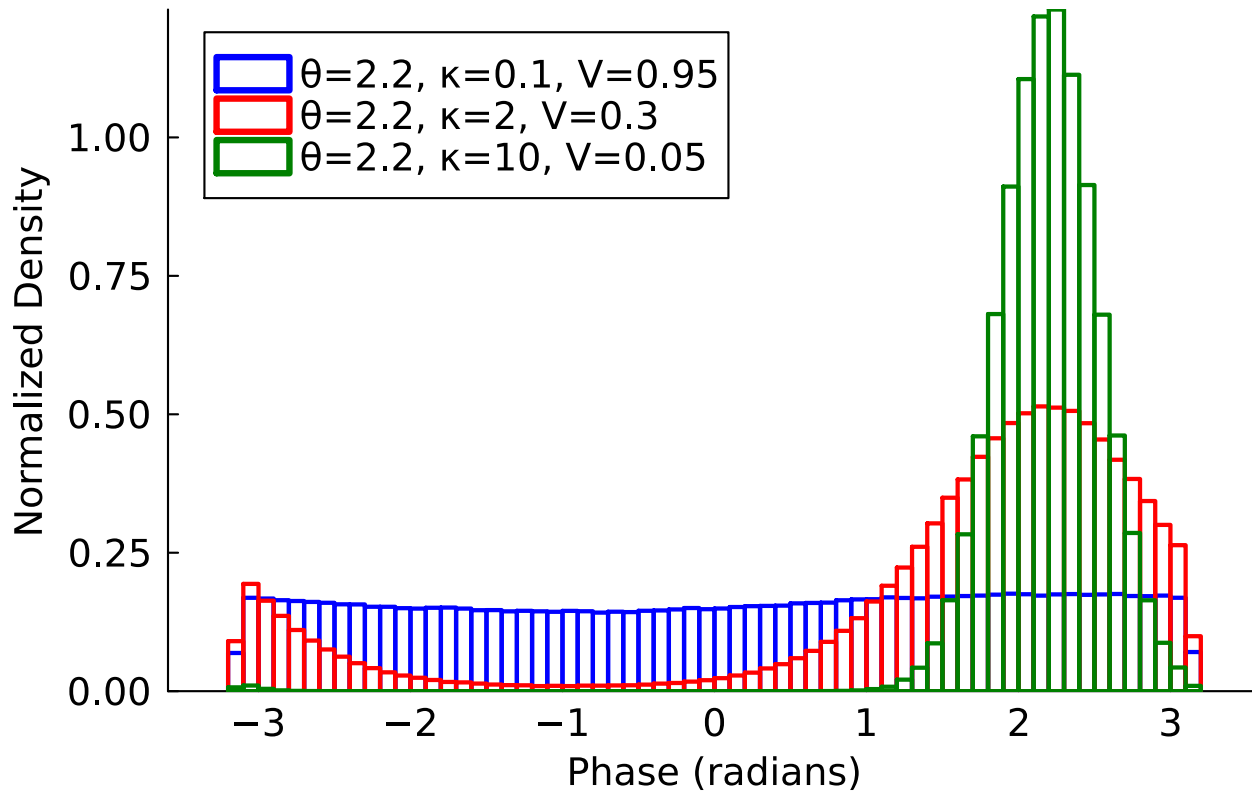
$$I_0(\kappa) = \sum_{m=0}^{\infty} \frac{1}{(m!)^2} \left(\frac{\kappa}{2}\right)^{2m}. \quad (12)$$

The von Mises distribution is unimodal and symmetric about its mean direction. When $\kappa = 0$, it reduces to the uniform distribution on $[-\pi, \pi]$, corresponding to completely random phases. As κ increases, the distribution becomes increasingly concentrated around $\bar{\theta}$ and approaches a normal distribution in shape, while preserving phase periodicity. Figure 4a illustrates this behavior for a fixed mean direction of 2.2 radians and increasing κ , showing the transition from nearly uniform to strongly clustered phase distributions.

The concentration parameter κ can be estimated from the mean resultant length $\bar{R}$ using standard approximations (Fisher, 1993),

$$\kappa \approx \begin{cases} 2\bar{R} + \bar{R}^3 + \frac{5\bar{R}^5}{6}, & \bar{R} < 0.53 \\ -0.4 + 1.39\bar{R} + \frac{0.43}{1 - \bar{R}}, & 0.53 \leq \bar{R} < 0.85 \\ \frac{1}{3\bar{R} - 4\bar{R}^2 + \bar{R}^3}, & \bar{R} \geq 0.85 \end{cases} \quad (13)$$

von Mises Distribution




```

1 begin
2     function wrap_phase(unwrapped_phase)
3         return mod.(unwrapped_phase .+ π, 2π) .- π
4     end
5
6     mean_value = 2.2
7
8     kappa1 = 0.1
9     x_vm1 = wrap_phase(rand(VonMises(mean_value, kappa1), 1000000))
10    V_vm1 = round(1 - circ_r(x_vm1), digits=2)
11
12    kappa2 = 2
13    x_vm2 = wrap_phase(rand(VonMises(mean_value, kappa2), 1000000))
14    V_vm2 = round(1 - circ_r(x_vm2), digits=2)
15
16    kappa3 = 10
17    x_vm3 = wrap_phase(rand(VonMises(mean_value, kappa3), 1000000))
18    V_vm3 = round(1 - circ_r(x_vm3), digits=2)
19
20    p4a = histogram(x_vm1, bins=100, normalize=true,
21                    linecolor=:blue, linewidth=2, fillalpha=0,
22                    label="θ=2.2, κ=$kappa1, V=$V_vm1",
23                    xlabel="Phase (radians)", ylabel="Normalized Density",
24                    title="von Mises Distribution",
25                    grid=false, legend=:topleft,
26                    fontsize=12, labelfontsize=12,
27                    legendfontsize=12, tickfontsize=12,
28                    xguidefontsize=12, yguidefontsize=12,
29                    dpi=300)
30
31    histogram!(p4a, x_vm2, bins=100, normalize=true,
32               linecolor=:red, linewidth=2, fillalpha=0,
33               label="θ=2.2, κ=$kappa2, V=$V_vm2")
34
35    histogram!(p4a, x_vm3, bins=100, normalize=true,
36               linecolor=:green, linewidth=2, fillalpha=0,
37               label="θ=2.2, κ=$kappa3, V=$V_vm3")
38
39    p4a
40 end

```

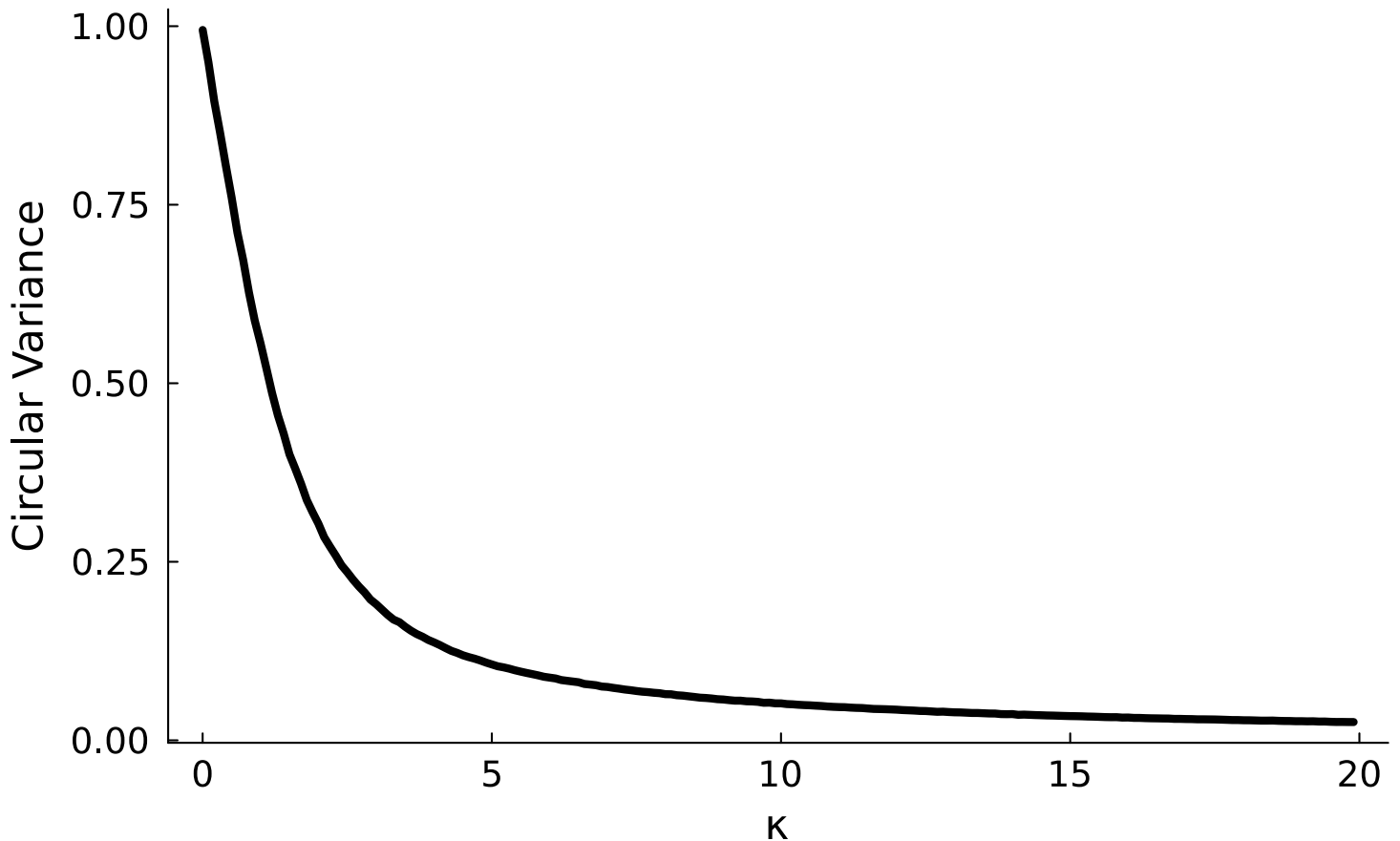
Circular variance V and the von Mises concentration parameter κ both describe the dispersion of phase vectors around the mean direction, but from complementary perspectives. Circular variance is defined directly from the mean resultant length $V = 1 - \bar{R}$ and provides a bounded, geometrically intuitive measure of phase variability, where values near zero indicate strong phase coherence and values near one indicate incoherent or noise-dominated behavior. In contrast, κ is specific to the von Mises model and quantifies the strength of clustering in an unbounded manner, ranging from zero to infinity. As a result, κ can be more sensitive to noise and may fluctuate strongly between neighboring ensembles, particularly at low coherence. When the von Mises model is assumed, however, V and κ are uniquely related through $\bar{R}$ and can be converted analytically or numerically, as illustrated in

Figure 4b. For clarity, Table 1 summarizes the key circular statistical quantities used in this study and their physical interpretation.

Attribute	Symbol	Range	Interpretation
Circular mean	$\bar{\theta}$	$[-\pi, \pi]$	Mean (central) phase direction
Mean resultant length	$\bar{R}$	$[0, 1]$	Degree of phase alignment (clustering strength)
Circular variance	$V = 1 - \bar{R}$	$[0, 1]$	Phase dispersion or incoherence
Concentration (von Mises)	κ	$[0, \infty)$	Phase clustering strength (inverse dispersion)

Table 1: Summary of key circular statistical attributes used for ensemble-based seismic phase analysis.

Circular Variance vs κ



```
1 begin
2   kappa_values = 0.001:0.1:20
3   circ_var_values = map( $\kappa$  -> 1.0 - circ_r(wrap_phase.(rand(VonMises(0.0,  $\kappa$ ),
4     100_001))), kappa_values)
5   p4b = plot(kappa_values, circ_var_values,
6     xlabel=" $\kappa$ ", ylabel="Circular Variance",
7     title="Circular Variance vs  $\kappa$ ",
8     legend=false, linewidth=4, color=:black, grid=false,
9     tickfontsize=12, xguidefontsize=14, yguidefontsize=14,
10    titlefontsize=16, dpi=300, size=(700, 450))
11
12   p4b
13 end
```

QUANTIFYING VARIABILITY IN SEISMIC PHASE

Although phase statistics are estimated directly from the data, without fitting any specific parametric distribution, the von Mises distribution remains an important interpretive reference. When a single coherent reflection dominates a local ensemble, the phase distribution is expected to be unimodal, with the mean direction representing the signal phase and the dispersion reflecting noise. This

provides a simple and intuitive signal-plus-noise picture in circular phase space.

Real seismic data are more complex. Noise can itself be partially coherent, such as ground roll, guided waves, or other organized wave modes, and multiple signals may interfere or cross within the same analysis window. In such cases, phase distributions may broaden asymmetrically or become multimodal. This behavior does not merely indicate stronger noise, but rather the absence of a single dominant signal phase. Importantly, the apparent phase distribution may depend on the number of traces in the ensemble: small local ensembles can appear unstable or noisy, whereas larger ensembles tend to reveal more stable phase structure. Even under these conditions, the unimodal von Mises case remains a useful reference state, providing a baseline for distinguishing signal-dominated behavior from mixed or incoherent phase regimes.

Building on this conceptual framework, we demonstrate how circular variance can be used as a quantitative quality-control attribute for seismic phase. Circular variance measures the spread of phase distributions within a local ensemble. Because seismic phase variability depends on both offset and frequency, these statistics must be computed locally using sliding windows over ensembles of traces. This approach enables objective tracking of phase variability across a seismic gather and provides a means to assess the impact of processing steps.

The workflow for computing circular variance is summarized as follows. Each local time-space window is analyzed in the frequency domain using the following procedure. Each trace $x_k(t)$ within the window is transformed to the frequency domain,

$$X_k(\omega) = \mathcal{F}\{x_k(t)\} = |X_k(\omega)| e^{i\theta_k(\omega)},$$

where $|X_k(\omega)|$ is the spectral amplitude and $\theta_k(\omega)$ the spectral phase.

- To isolate phase information, each spectrum is normalized to unit magnitude,

$$\tilde{X}_k(\omega) = \frac{X_k(\omega)}{|X_k(\omega)|} = e^{i\theta_k(\omega)}.$$

- The set $\{\tilde{X}_k(\omega)\}_{k=1}^N$ thus forms a phase ensemble at frequency ω .
- Phase dispersion within the local ensemble is quantified using the circular variance,

$$V(\omega) = 1 - \left| \frac{1}{N} \sum_{k=1}^N e^{i\theta_k(\omega)} \right|,$$

where $V(\omega) \in [0, 1]$.

- The phase variance $V(\omega)$ is assigned to the center of the analysis window and mapped across time and frequency.

The ensemble size N is chosen to ensure statistically stable and interpretable estimates, as discussed in Appendix A. To assess the reliability of this metric, we first apply it to a controlled synthetic example. This allows us to demonstrate how circular variance responds to known phase perturbations, providing a benchmark before extending the method to more complex field data.

Synthetic example with phase perturbations

The choice of ensemble size is dictated by the role of the analysis and is kept consistent between synthetic tests and field applications. In high-noise conditions, larger ensembles are required to see through noise and obtain statistically stable estimates, whereas in low-noise settings smaller ensembles may suffice. Accordingly, large ensembles of 10,000 traces analyzed using 2,000-trace sliding windows are employed to obtain statistically stable estimates of circular variance when it is used as a diagnostic and QC attribute. This ensures that the measured phase variability reflects the imposed perturbations rather than sampling noise.

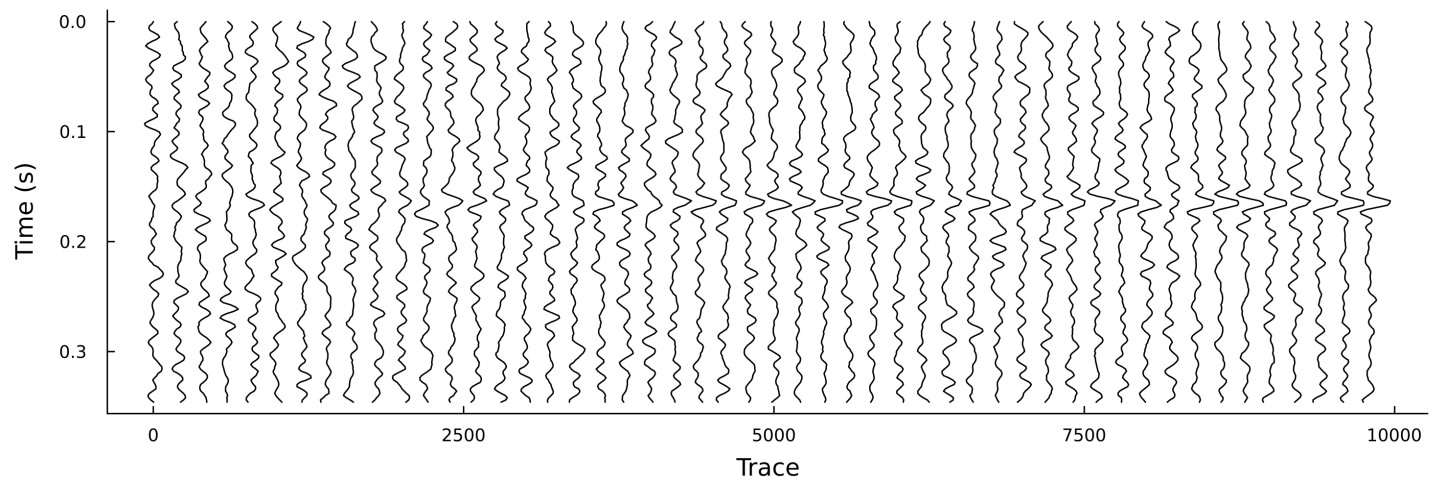
To investigate the variability of seismic phase within an ensemble, we designed a controlled synthetic experiment. Phase perturbations were introduced by drawing random realizations from a von Mises distribution, consistent with the conceptual model described earlier. For each trace, a circular variance V was prescribed and systematically varied across the ensemble to impose laterally varying phase noise. Figure 5a illustrates the imposed variation in phase perturbations as a function of trace number. The variance decreases from left to right, mimicking a realistic acquisition scenario in which near-offset traces fall within a noise cone characterized by stronger phase perturbations, while far-offset traces exhibit increasingly coherent reflection behavior. The resulting synthetic seismic section is shown in Figure 5b.

offset_dependent_perturbation (generic function with 1 method)

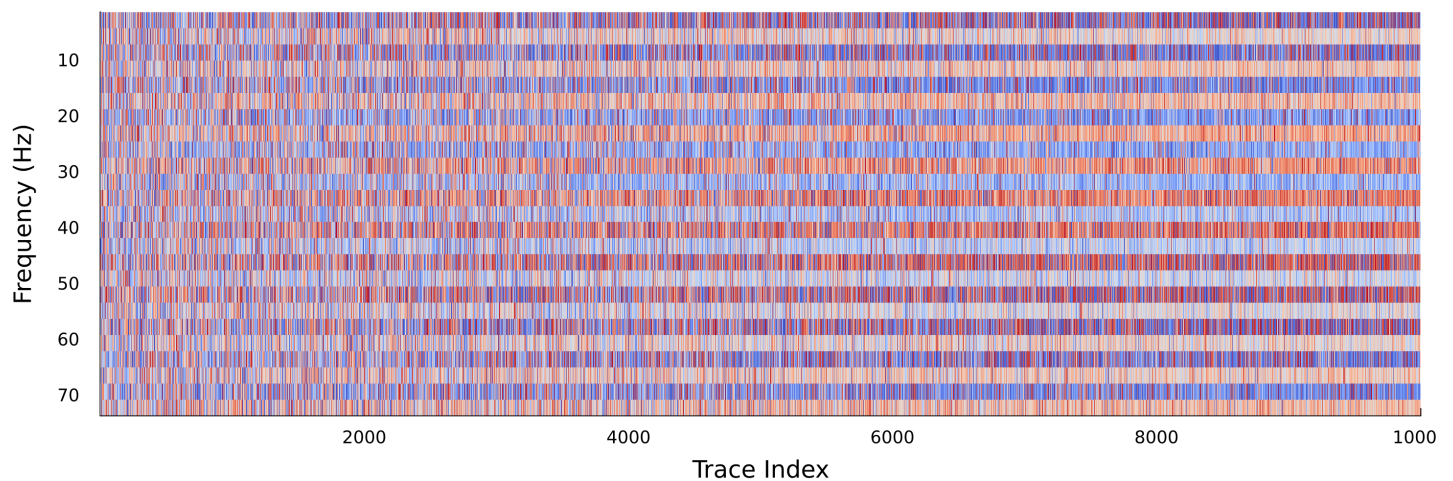
```
1 function offset_dependent_perturbation(phase_clean, amplitude_clean, f_zero, t,  
   num_wiggles;  
2                                     kappa_start=0.2, kappa_stop=2.0)  
3     Random.seed!(2025)  
4     kappa_values = range(start=kappa_start, stop=kappa_stop, length=num_wiggles)  
5     var_values   = [1 - besseli(1, k)/besseli(0, k) for k in kappa_values]  
6     phase_perturbed = similar(phase_clean)  
7     for i in 1:size(phase_clean, 2)  
8         pp      = phase_clean[:, i]  
9         kappat = kappa_values[i]  
10        random = rand(VonMises(0.0, kappat), size(phase_clean, 1))  
11        phase_perturbed[:, i] .= pp .+ random  
12    end  
13    phase_perturbed = mod.(phase_perturbed .+  $\pi$ , 2 $\pi$ ) .-  $\pi$   
14    new_f = amplitude_clean .* exp.(1.0im .* phase_perturbed)  
15    perturbed_signal_half = zeros(ComplexF64, size(new_f, 1), num_wiggles)  
16    for i in 1:size(new_f, 2)  
17        perturbed_signal_half[:, i] = reverse(conj(new_f[:, i]))  
18    end  
19    perturbed_signal = vcat(f_zero, new_f, perturbed_signal_half)  
20    ifft_perturbed = zeros(Float64, size(perturbed_signal, 1), num_wiggles)  
21    for i in 1:size(perturbed_signal, 2)  
22        ifft_perturbed[:, i] = real(ifft(perturbed_signal[:, i]))  
23    end  
24    return ifft_perturbed, phase_perturbed, var_values  
25 end
```

```
► (173×10000 Matrix{Float64}:  
   -0.000489074  0.0220992 -0.203244 ...  0.0452056 -0.0245502 -0.229723  2.12363, 86×10000 M  
1 offset_traces, phase_offset, var_imposed = offset_dependent_perturbation(  
2   phase_clean, amplitude_clean, f_zero, t, num_wiggles;  
3   kappa_start=0.2, kappa_stop=2.0)
```

Noisy traces



Offset-dependent perturbed phases



```

1  begin
2      p5b = plot_wiggles(offset_traces, t; title="Noisy traces")
3
4      p5c = heatmap(1:10000, positive_freq[1:25], phase_offset[1:25, 1:10000],
5                    yflip=true, c=:coolwarm, colorbar=false, clims=(- $\pi$ ,  $\pi$ ),
6                    ylabel="Frequency (Hz)", title="Offset-dependent perturbed phases",
7                    xlabel="Trace Index")
8
9      l_5bc = @layout [a{0.5h}; b{0.5h}]
10
11     plot(p5b, p5c,
12           layout=l_5bc,
13           size=(1000, 800),
14           left_margin=8Plots.mm,
15           bottom_margin=8Plots.mm,
16           top_margin=4Plots.mm,
17           dpi=300)
18  end

```

The baseline signal was generated using a Klauder wavelet, defined as the autocorrelation of a

vibroseis sweep commonly employed in land seismic surveys. This wavelet served as a clean reference, from which an ensemble of 10,000 identical traces was constructed. In the frequency domain, the unperturbed signal at channel k and frequency ω is given by

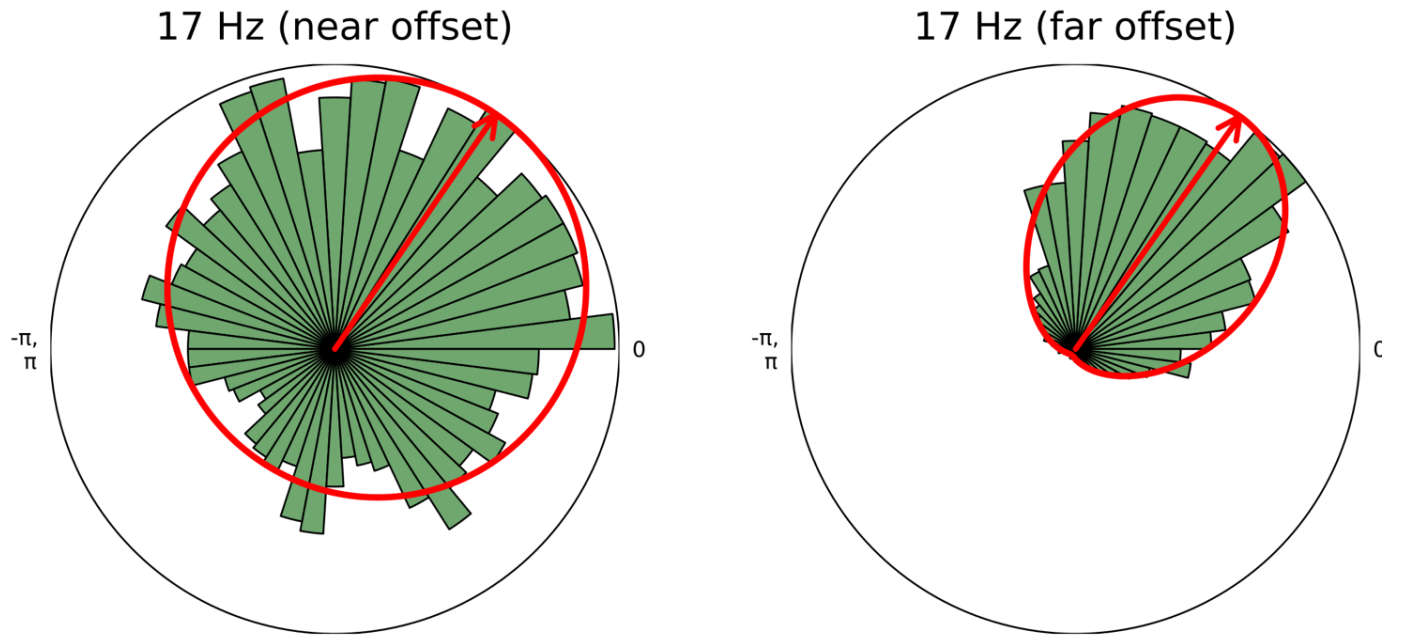
$$X_k(\omega) = A_0(\omega) e^{i\theta_0(\omega)}, \quad (14)$$

where $A_0(\omega)$ denotes the true amplitude spectrum and $\theta_0(\omega)$ the corresponding true phase. Phase distortions were then introduced by applying spatially dependent perturbations, such that the perturbed signal becomes

$$X_k(\omega) = A_0(\omega) e^{i\{\theta_0(\omega) + \theta_k^\delta(\omega)\}}, \quad (15)$$

where $\theta_k^\delta(\omega)$ represents the imposed perturbation.

Within each window, amplitudes were normalized to isolate phase effects, ensuring that the computed statistics reflect phase variability alone. The resulting phase ensemble in the frequency domain is shown in Figure 5c. Sliding the window across the ensemble allows phase distributions to be evaluated as a function of offset and frequency. Representative phase distributions extracted from near-offset and far-offset windows are shown in Figures 5e and 5f, respectively. These distributions illustrate the transition from highly dispersed phases at near offsets to more concentrated phase behavior at far offsets. For example, at 17 Hz the near-offset window yields a circular variance of 0.86, indicating strong phase variability, whereas the far-offset window produces a lower value of 0.32, reflecting greater phase stability.




```

1 begin
2   # circular stats for near and far offset at 17 Hz
3   mean_c_near_offset = circ_mean(phase_offset[6, 1:2000]).μ
4   kappa_c_near_offset = kappa_from_R(circ_r(phase_offset[6, 1:2000]))
5   mean_c_far_offset = circ_mean(phase_offset[6, 8000:10000]).μ
6   kappa_c_far_offset = kappa_from_R(circ_r(phase_offset[6, 8000:10000]))
7
8   function add_circular_mean_arrow!(p, μ; scale=1.0, color=:red, lw=3)
9     μw = mod(μ, 2π)
10    xlims_p = Plots.xlims(p)
11    R = maximum(abs.(xlims_p)) * scale
12    quiver!(p, [0.0], [0.0];
13             quiver=[R * cos(μw)], [R * sin(μw)],
14             color=color, lw=lw, arrow=true, label=false)
15  end
16
17  function add_circle_annotations!(p)
18    xlims_p = Plots.xlims(p)
19    R = maximum(abs.(xlims_p))
20    # -π, π on the left
21    annotate!(p, -R * 1.05, 0.0, text("-π,\nπ", :black, 8, :right))
22    # 0 on the right
23    annotate!(p, R * 1.05, 0.0, text("0", :black, 8, :left))
24  end
25
26  # 5e - near offset circular histogram
27  p5e = circ_hist_with_vm(phase_offset[6, 1:2000], 2π/50,
28                          mean_c_near_offset, kappa_c_near_offset;
29                          title_str="17 Hz (near offset)", show_vm=true)
30  add_circular_mean_arrow!(p5e, mean_c_near_offset)
31  add_circle_annotations!(p5e)
32
33  # 5f - far offset circular histogram
34  p5f = circ_hist_with_vm(phase_offset[6, 8000:10000], 2π/50,
35                          mean_c_far_offset, kappa_c_far_offset;
36                          title_str="17 Hz (far offset)", show_vm=true)
37  add_circular_mean_arrow!(p5f, mean_c_far_offset)
38  add_circle_annotations!(p5f)
39
40  plot(p5e, p5f,
41       layout=(1, 2),
42       size=(800, 400),
43       left_margin=8Plots.mm,
44       bottom_margin=8Plots.mm,
45       dpi=300)
46  end

```

The circular variance computed within each window is assigned to the window center before advancing to the next position. Repeating this procedure produces a new seismic attribute, referred to here as phase variance. The resulting phase-variance map (Figure 5d) captures the spatial and spectral evolution of phase variability across the synthetic gather. As expected, phase variance is highest at

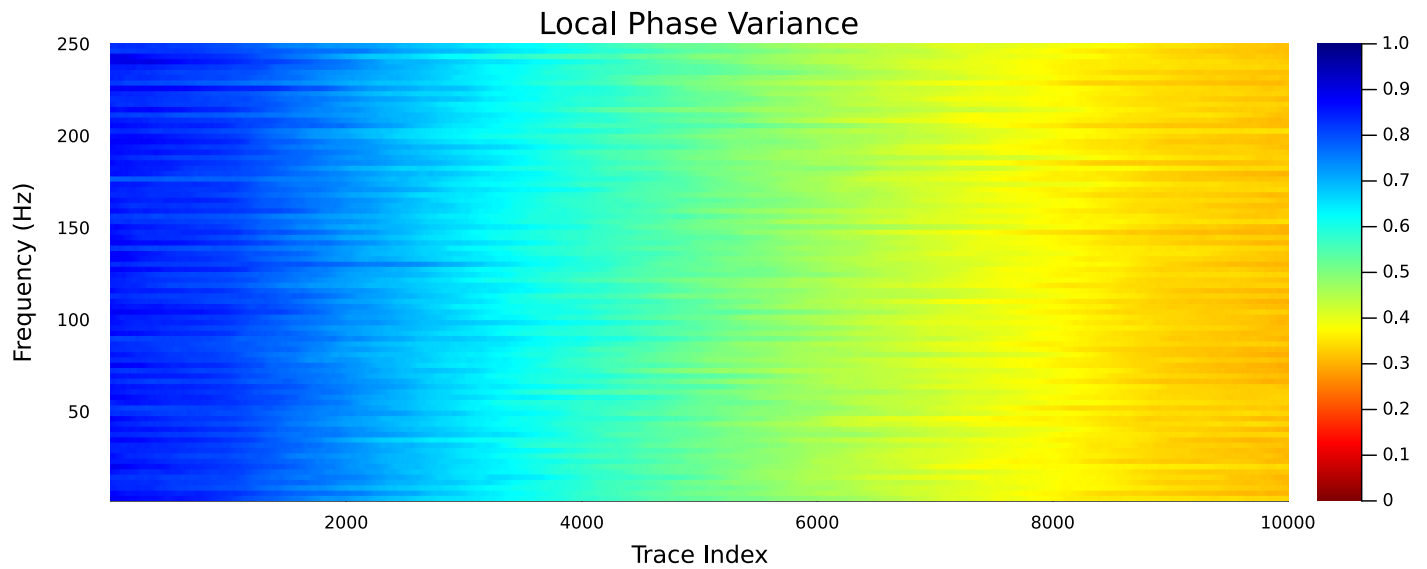
near offsets and decreases steadily toward far offsets, consistent with the imposed perturbations. Because the perturbations in this test were identical across frequencies, the frequency dependence of the phase-variance map is uniform. The framework naturally extends to cases where phase noise varies with frequency, although the present example focuses on spatial variability for clarity.

The average phase variance across all traces (Figure 5g) shows close agreement between the imposed and recovered values, confirming that circular variance reliably quantifies phase perturbations in a controlled setting. These synthetic results demonstrate that circular variance provides a robust and interpretable measure of seismic phase variability. Having validated that $V(\omega)$ accurately recovers known phase perturbations in synthetic data, we now apply it as a QC observable to field seismic gathers to diagnose where phase becomes unreliable.

```
local_phase_variance (generic function with 1 method)
1 function local_phase_variance(phases; batch_size=2000)
2     n_freqs, n_traces = size(phases)
3     half_batch = div(batch_size, 2)
4     variance = Matrix{Float64}(undef, n_freqs, n_traces)
5
6     for j in 1:n_traces
7         start_idx = max(1, j - half_batch)
8         end_idx = min(n_traces, j + half_batch - 1)
9
10        for i in 1:n_freqs
11            window_phases = phases[i, start_idx:end_idx]
12            r = circ_r(window_phases)
13            variance[i, j] = 1 - r
14        end
15    end
16
17    return variance
18 end
```

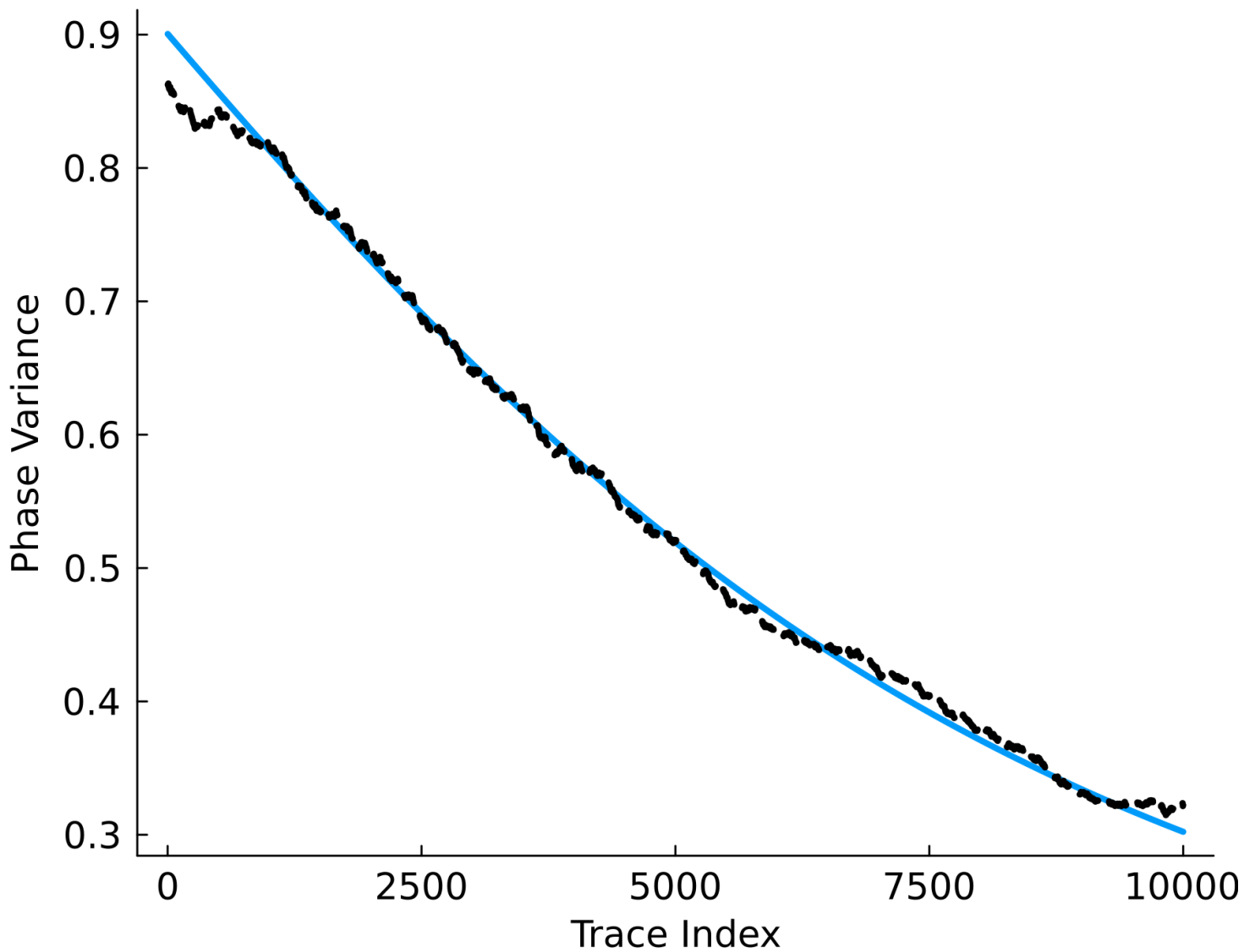
```
1 variance_offset = local_phase_variance(phase_offset; batch_size=2000);
```

p5d =



```
1 p5d = heatmap(1:10000, positive_freq[1:size(variance_offset, 1)], variance_offset,  
2             yflip=false, c=cgrad(:jet, rev=true), colorbar=true, clim=(0, 1),  
3             ylabel="Frequency (Hz)", title="Local Phase Variance",  
4             xlabel="Trace Index", size=(1000,400),  
5             left_margin=8Plots.mm, bottom_margin=8Plots.mm, dpi=300)
```

True vs Estimated Phase Variance



```
1 begin
2   p5g = plot(var_imposed, label="Variance in", lw=3, grid=false)
3   plot!(p5g, variance_offset[6, :], label="Variance out", lw=3,
4         linestyle=:dash, color=:black,
5         ylabel="Phase Variance", xlabel="Trace Index",
6         title="True vs Estimated Phase Variance",
7         fontsize=12, labelfontsize=12,
8         legendfontsize=12, tickfontsize=12,
9         xguidefontsize=12, yguidefontsize=12,
10        size=(600, 500), grid=false, dpi=300)
11
12   p5g
13 end
```

Monitoring phase variance in field data

Monitoring phase quality is critical in seismic processing because phase integrity directly controls imaging, inversion, and attribute analysis. Despite its importance, conventional workflows provide few tools for quantifying phase variability or tracking phase noise in a systematic way. As a result, phase distortions caused by near-surface effects, scattering, or processing artifacts may persist and propagate through the workflow without being explicitly diagnosed. Circular variance provides a practical phase quality-control attribute that allows phase behavior to be evaluated objectively as a function of offset and frequency.

To illustrate this capability, we analyze two prestack seismic gathers extracted from a 3D CMP supergather, one before and one after conventional time processing. A 300-ms time window containing a deep reflection event was selected. Phase statistics were computed using a sliding-window approach with local ensembles of 2,000 traces across a total of 10,000 traces. The gathers shown in Figures 6a and 6b are displayed after normal moveout correction and statics. The conventional time-processing sequence included linear noise attenuation, refraction statics, random noise removal, two passes of surface-consistent deconvolution, post-deconvolution noise attenuation targeting linear, random, and burst noise, merge-phase and static matching, surface-consistent scaling, velocity analysis, and residual statics correction.

The effect of processing on seismic phase can be examined by inspecting phase distributions within local time–frequency windows. At 16 Hz in a mid-offset local window, the pre-processed data exhibit a broad and weakly organized phase distribution, indicative of substantial phase variability (Figure 6c). After conventional processing, the same window shows noticeably tighter phase clustering (Figure 6d), with circular variance decreasing from 0.81 to 0.59. This reduction reflects improved phase coherence at that frequency and offset and demonstrates how phase variance provides a quantitative measure of processing impact on seismic phase behavior in field data.

```
read_bin_seismic (generic function with 1 method)
```

```
1 function read_bin_seismic(file_id, rows, cols, ::Type{T}) where {T}
2     url = "https://drive.google.com/uc?export=download&id=$file_id"
3     response = HTTP.get(url)
4     data = Matrix{T}(undef, rows, cols)
5     read!(IOBuffer(response.body), data)
6     return data
7 end
```

```
trace_normalize (generic function with 1 method)
```

```
1 function trace_normalize(data::Matrix{T}) where {T <: AbstractFloat}
2     normalized = similar(data)
3     for i in 1:size(data, 2)
4         max_val = maximum(abs.(data[:, i]))
5         normalized[:, i] .= max_val != 0 ? data[:, i] ./ max_val : data[:, i]
6     end
7     return normalized
8 end
```

```
1 raw_supergather = read_bin_seismic("1NZ1hvEVtE7rhHpaMAmbeKTeiAUZuj47y", 149, 10000,  
2 Float32);
```

```
1 processed_supergather = read_bin_seismic("1bPv6pk3eYzkHKU4tIiXmbGr9Uc26bIm4", 149,  
2 10000, Float32);
```

```
t_seismic = 0.0:0.002:0.296
```

```
1 t_seismic = range(start=0, step=0.002, length=149)
```

```
► (74×10000 Matrix{Float32}):  
  0.0158646  0.148426  -2.63288  -0.5484  ...  -0.0383151  -2.63223  -0.496718  1392
```

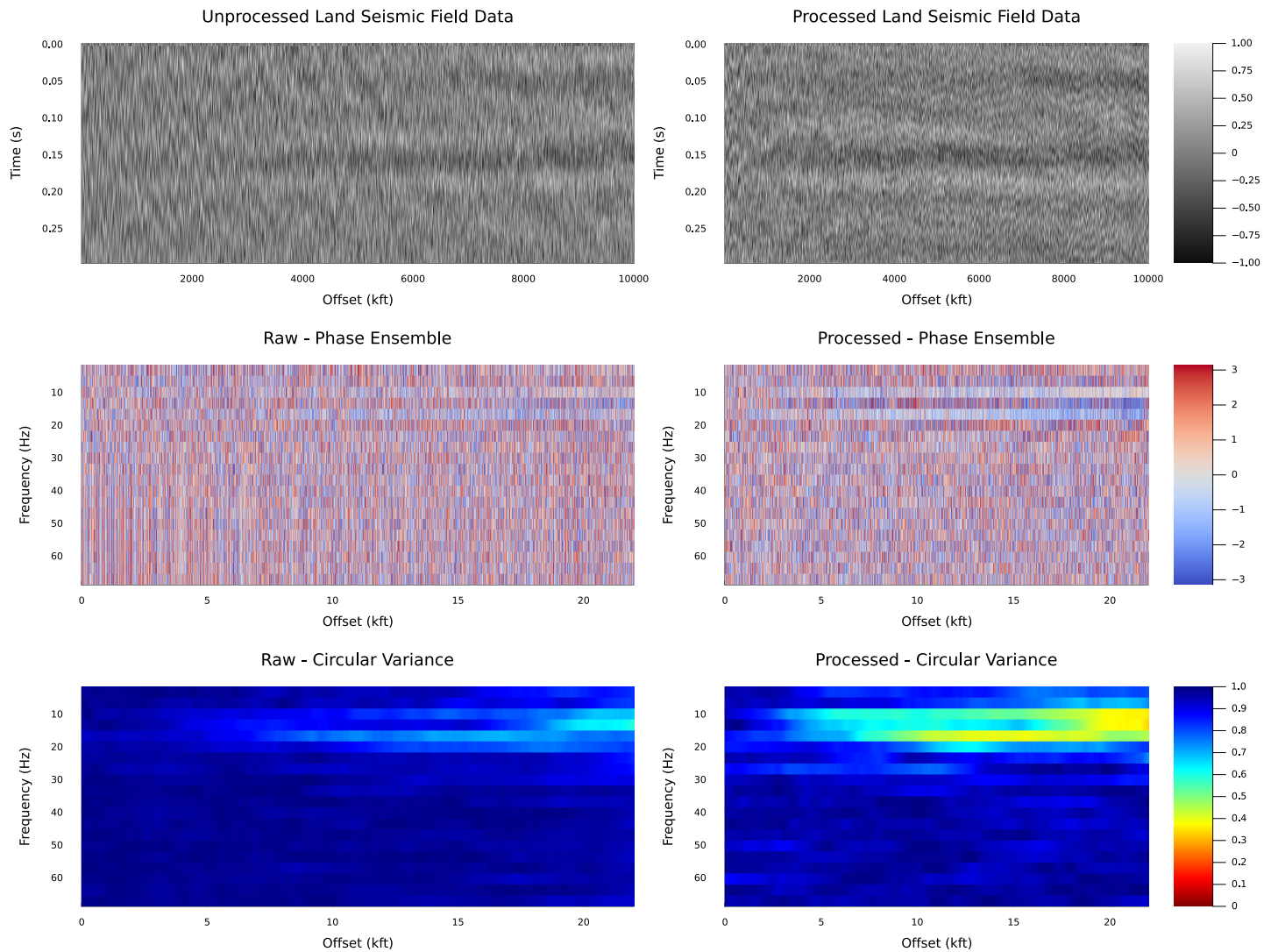
```
1 phase_raw, amplitude_raw, positive_freq_raw, f_zero_raw =  
2 trace_spectrum(raw_supergather, T, extras=true)
```

```
1 variance_raw = local_phase_variance(phase_raw; batch_size=2000);
```

```
► (74×10000 Matrix{Float32}):  
 -1.43339  -1.57452  -1.11875  -1.42312  ...  -0.529132  -3.03848  1.00964
```

```
1 phase_processed, amplitude_processed, positive_freq_processed, f_zero_processed =  
2 trace_spectrum(processed_supergather, T, extras=true)
```

```
1 variance_processed = local_phase_variance(phase_processed; batch_size=2000);
```



```

1 begin
2     n_traces_field = size(raw_supergather, 2)
3     offset_axis = range(0, stop=22, length=n_traces_field)
4     raw_norm = trace_normalize(raw_supergather)
5     processed_norm = trace_normalize(processed_supergather)
6
7     # — 6a — raw seismic heatmap
8     p6a = heatmap(1:size(raw_norm, 2), t_seismic, raw_norm,
9                   yflip=true, c=:grays, colorbar=false,
10                  ylabel="Time (s)", xlabel="Offset (kft)",
11                  title="Unprocessed Land Seismic Field Data")
12
13     # — 6b — processed seismic heatmap
14     p6b = heatmap(1:size(processed_norm, 2), t_seismic, processed_norm,
15                  yflip=true, c=:grays, colorbar=true,
16                  ylabel="Time (s)", xlabel="Offset (kft)",
17                  title="Processed Land Seismic Field Data")
18
19     # — 6c — raw phase heatmap
20     p6c = heatmap(offset_axis, positive_freq_raw[1:20],
21                   phase_raw[1:20, 1:n_traces_field],
22                   yflip=true, c=:coolwarm, colorbar=false, clims=(- $\pi$ ,  $\pi$ ),
23                   ylabel="Frequency (Hz)", xlabel="Offset (kft)",

```

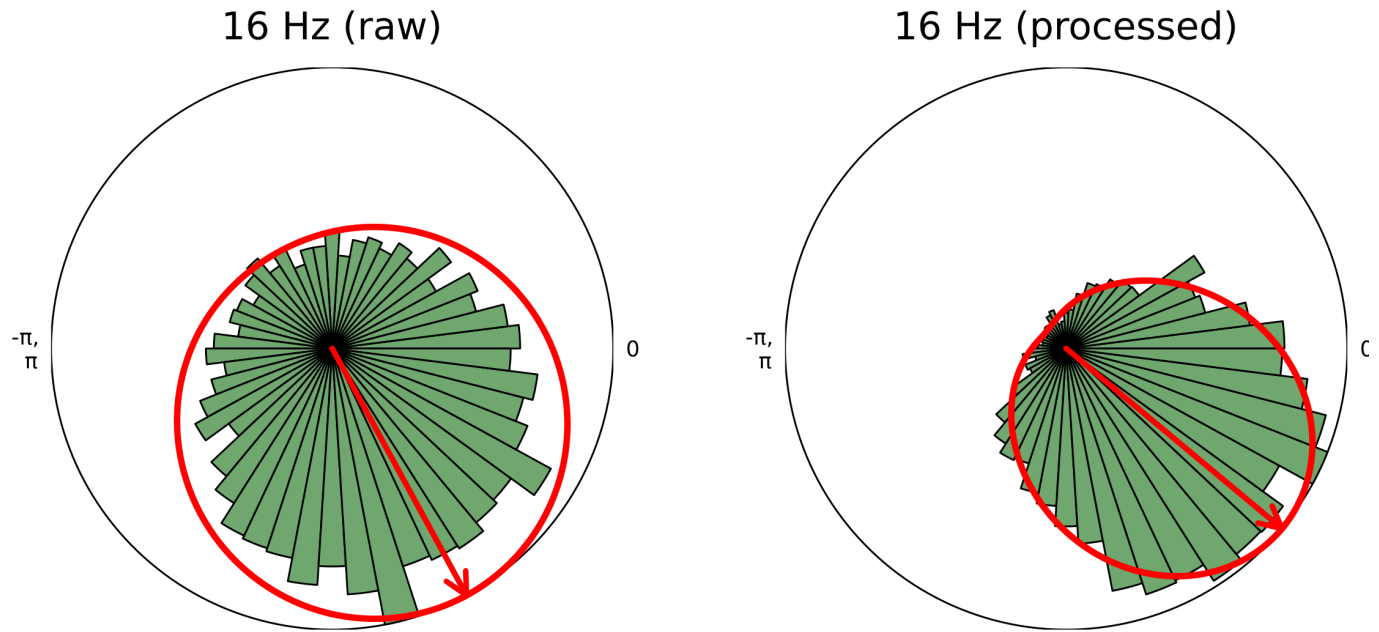
```

24         title="Raw - Phase Ensemble")
25
26     # — 6d - processed phase heatmap
27     p6d = heatmap(offset_axis, positive_freq_processed[1:20],
28                   phase_processed[1:20, 1:n_traces_field],
29                   yflip=true, c=:coolwarm, colorbar=true, clims=(- $\pi$ ,  $\pi$ ),
30                   ylabel="Frequency (Hz)", xlabel="Offset (kft)",
31                   title="Processed - Phase Ensemble")
32
33     # — 6g - raw phase variance heatmap
34     p6g = heatmap(offset_axis, positive_freq_raw[1:20],
35                   variance_raw[1:20, :],
36                   yflip=true, c=cgrad(:jet, rev=true), colorbar=false, clims=(0, 1),
37                   ylabel="Frequency (Hz)", xlabel="Offset (kft)",
38                   title="Raw - Circular Variance")
39
40     # — 6h - processed phase variance heatmap
41     p6h = heatmap(offset_axis, positive_freq_processed[1:20],
42                   variance_processed[1:20, :],
43                   yflip=true, c=cgrad(:jet, rev=true), colorbar=true, clims=(0, 1),
44                   ylabel="Frequency (Hz)", xlabel="Offset (kft)",
45                   title="Processed - Circular Variance")
46
47     # — layout
48     l_fig6 = @layout [
49         a b
50         c d
51         e f
52     ]
53
54     plot(p6a, p6b, p6c, p6d, p6g, p6h,
55          layout=l_fig6,
56          size=(1600, 1200),
57          left_margin=8Plots.mm,
58          bottom_margin=8Plots.mm,
59          top_margin=4Plots.mm,
60          dpi=300)
61 end

```

To generalize these observations, we computed circular variance across all frequencies and offsets using overlapping sliding time windows (Figures 6g and 6h). Although reflectors appear visually clearer after conventional processing, the phase-variance maps reveal a more nuanced picture. At higher frequencies, circular variance remains close to one, indicating that phase behavior is still dominated by noise despite apparent amplitude enhancement. The spatial distribution of variance highlights systematic trends. Near offsets, which fall within the noise cone, exhibit consistently high phase variance, while mid to far offsets show progressively lower variance. For example, at 16 Hz, the unprocessed near-offset data yield a variance of 0.93, characteristic of noise-dominated phase behavior, whereas mid to far offsets decrease to about 0.71, indicating partial coherence. After processing, these same offsets show a marked reduction in variance to approximately 0.40,

quantitatively confirming that conventional processing improves phase coherence primarily at lower frequencies and away from the noise cone (Figures 6j and 6l).

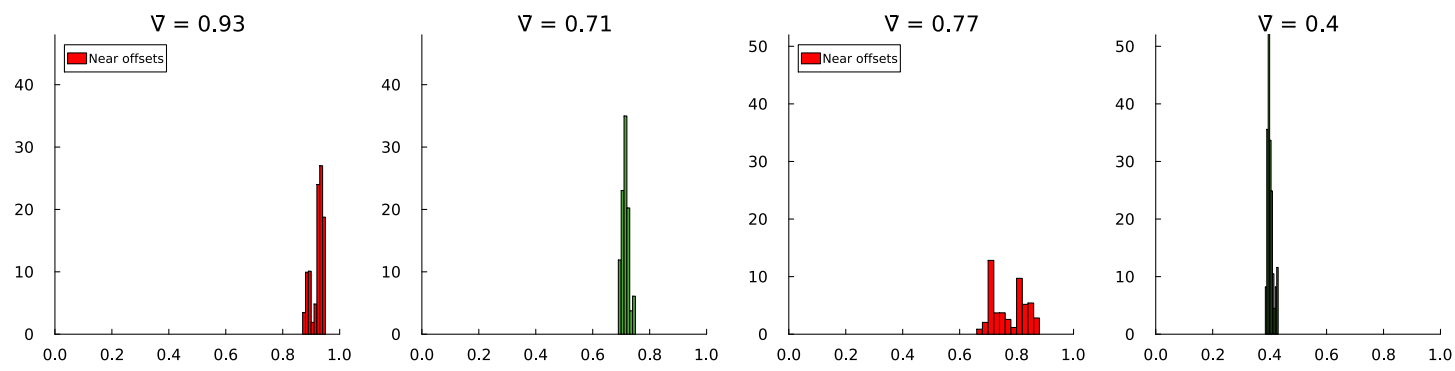


```

1 begin
2   # circular stats for raw and processed at 16 Hz (freq index 5)
3   mean_c_16_raw      = circ_mean(phase_raw[5, 2500:7500]).μ
4   kappa_c_16_raw     = kappa_from_R(circ_r(phase_raw[5, 2500:7500]))
5   mean_c_16_processed = circ_mean(phase_processed[5, 2500:7500]).μ
6   kappa_c_16_processed = kappa_from_R(circ_r(phase_processed[5, 2500:7500]))
7
8   # 6c inset - raw circular histogram
9   p6c_circ = circ_hist_with_vm(phase_raw[5, 2500:7500], 2π/50,
10                                mean_c_16_raw, kappa_c_16_raw;
11                                title_str="16 Hz (raw)", show_vm=true)
12   add_circular_mean_arrow!(p6c_circ, mean_c_16_raw)
13   add_circle_annotations!(p6c_circ)
14
15   # 6d inset - processed circular histogram
16   p6d_circ = circ_hist_with_vm(phase_processed[5, 2500:7500], 2π/50,
17                                mean_c_16_processed, kappa_c_16_processed;
18                                title_str="16 Hz (processed)", show_vm=true)
19   add_circular_mean_arrow!(p6d_circ, mean_c_16_processed)
20   add_circle_annotations!(p6d_circ)
21
22   plot(p6c_circ, p6d_circ,
23         layout=(1, 2),
24         size=(800, 400),
25         left_margin=8Plots.mm,
26         bottom_margin=8Plots.mm,
27         dpi=300)
28 end

```

This trend is consistent with stack-based signal-to-noise ratio estimates (Figure 7a), computed following Bakulin et al. (2022b). Prestack SNR improves from roughly **−22** dB to **−12** dB after processing, reflecting substantial suppression of random noise. However, SNR averages energy over the full frequency band and therefore masks important broadband behavior. In contrast, circular variance isolates phase variability directly and reveals that improvements are confined to a limited frequency range.



```

1 begin
2   # — raw stats
3   mean_var_near_raw = round(mean(vec(variance_raw[5, 1:2000])), digits=2)
4   mean_var_far_raw  = round(mean(vec(variance_raw[5, 6000:8000])), digits=2)
5
6   p6i = histogram(vec(variance_raw[5, 1:3000]),
7                   label="Near offsets", title="V̄ = $mean_var_near_raw",
8                   fill="#FF0000", bins=10, grid=false, normalize=true,
9                   legend=:topleft,
10                  xtickfontsize=10, ytickfontsize=10)
11   xlims!(p6i, 0.0, 1.0); ylims!(p6i, 0, 48)
12
13   p6j = histogram(vec(variance_raw[5, 6000:8000]),
14                   label="Far offsets", title="V̄ = $mean_var_far_raw",
15                   fill="#4EA72E", bins=10, grid=false, normalize=true,
16                   xtickfontsize=10, ytickfontsize=10)
17   xlims!(p6j, 0.0, 1.0); ylims!(p6j, 0, 48)
18
19   # — processed stats
20   mean_var_near_processed = round(mean(vec(variance_processed[5, 1:3000])),
21   digits=2)
22   mean_var_far_processed  = round(mean(vec(variance_processed[5, 6000:8000])),
23   digits=2)
24
25   p6k = histogram(vec(variance_processed[5, 1:3000]),
26                   label="Near offsets", title="V̄ = $mean_var_near_processed",
27                   fill="#FF0000", bins=10, grid=false, normalize=true,
28                   legend=:topleft,
29                   xtickfontsize=10, ytickfontsize=10)
30   xlims!(p6k, 0.0, 1.0); ylims!(p6k, 0, 52)
31
32   p6l = histogram(vec(variance_processed[5, 6000:8000]),
33                   label="Far offsets", title="V̄ = $mean_var_far_processed",
34                   fill="#4EA72E", bins=10, grid=false, normalize=true,
35                   xtickfontsize=10, ytickfontsize=10)
36   xlims!(p6l, 0.0, 1.0); ylims!(p6l, 0, 52)
37
38   plot(p6i, p6j, p6k, p6l,
39        layout=(1, 4),
40        size=(1400, 350),
41        left_margin=6Plots.mm,
42        bottom_margin=6Plots.mm,
43        dpi=300)
44 end

```

Amplitude spectra before and after processing (Figure 7b) show that conventional workflows often boost high frequencies through repeated deconvolution and filtering, giving the appearance of increased bandwidth. Amplitude-based measures alone, however, do not capture phase stability and can therefore be misleading. Phase-variance spectra (Figure 7c) provide a complementary and more physically meaningful perspective. By examining circular variance as a function of frequency, averaged over offsets, it becomes possible to define an effective frequency band in terms of phase coherence

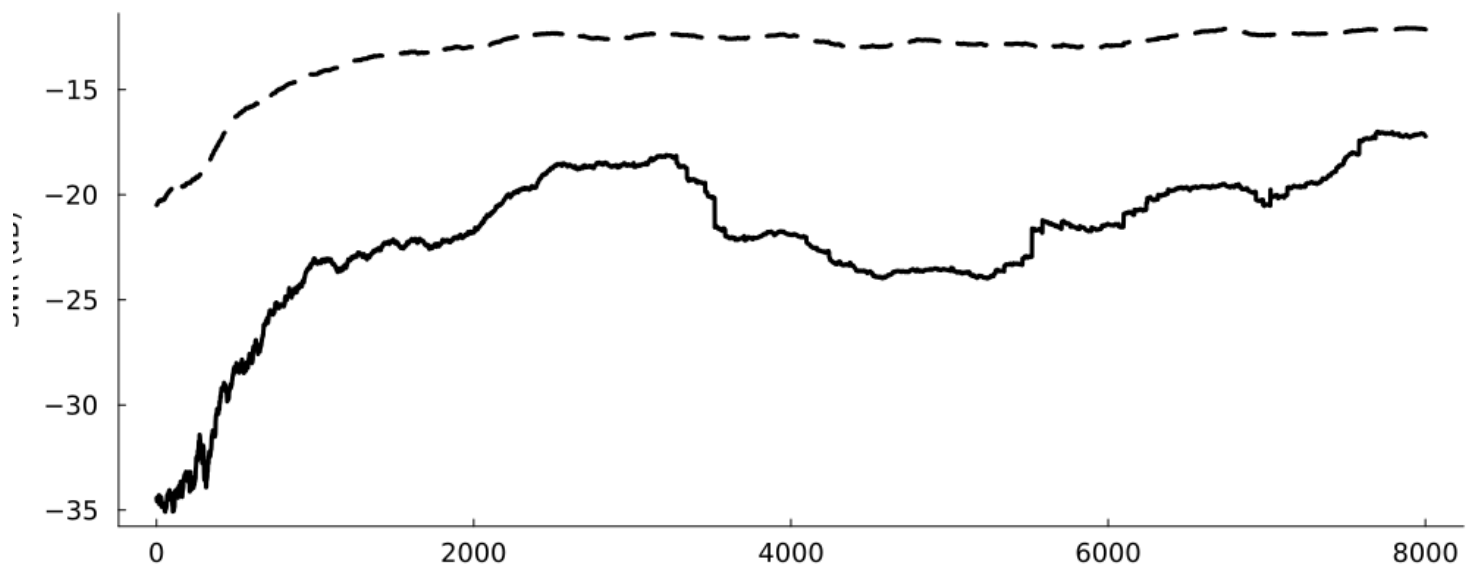
rather than amplitude alone. In practice, one can select a threshold variance corresponding to acceptable phase stability for a given application. Frequencies below this threshold retain coherent phase and are suitable for phase-sensitive workflows such as full-waveform inversion and AVO analysis, whereas frequencies above it contribute primarily incoherent energy.

compute_snr (generic function with 1 method)

```
1 function compute_snr(data; step_size=1, xdims=2000, ydims=149)
2     rows = size(data, 1)
3     cols = size(data, 2)
4     snr_values = Float64[]
5     for i in 1:step_size:rows-ydims+1
6         for j in 1:step_size:cols-xdims+1
7             window = data[i:i+ydims-1, j:j+xdims-1]
8             sem = calculate_sembance(window)
9             snr = sem / (1 - sem)
10            snr_db = 10 * log10(snr)
11            push!(snr_values, snr_db)
12        end
13    end
14    return snr_values
15 end
```

calculate_sembance (generic function with 1 method)

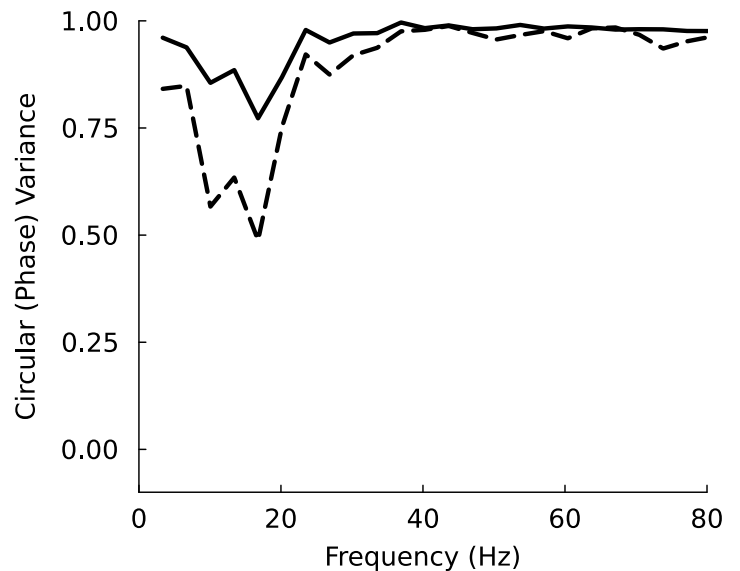
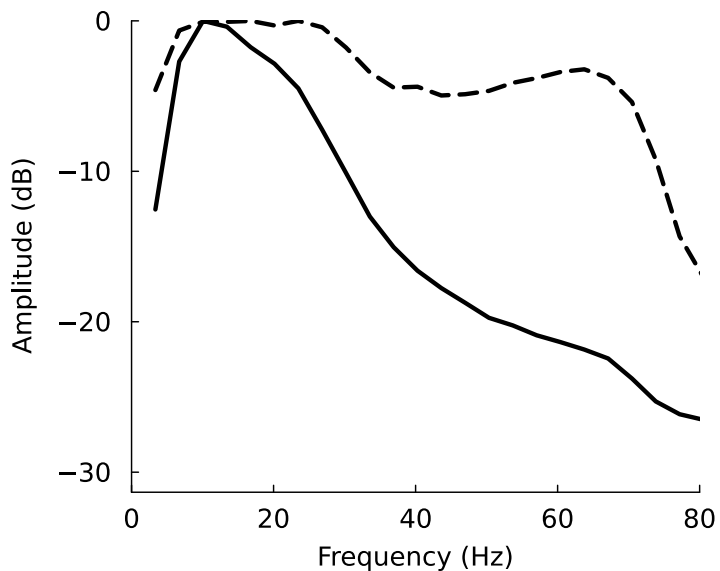
```
1 function calculate_sembance(window)
2     N, M = size(window)
3     numerator = sum((sum(window[i, j] for j in 1:M))^2 for i in 1:N)
4     denominator = M * sum(sum(window[i, j]^2 for j in 1:M) for i in 1:N)
5     return numerator / denominator
6 end
```



```

1 begin
2     SNR_value_raw      = compute_snr(raw_supergather)
3     SNR_value_processed = compute_snr(processed_supergather)
4
5     p7a = plot(SNR_value_raw, lw=3, label="Unprocessed", color=:black,
6               ylabel="SNR (dB)", xlabel="Trace Index", grid=false,
7               fontsize=12, xtickfontsize=12, ytickfontsize=12,
8               guidefontsize=12, legendfontsize=12, size=(1000, 400))
9     plot!(p7a, SNR_value_processed, lw=3, label="Processed",
10           color=:black, linestyle=:dash)
11
12     p7a
13 end

```



```

1 begin
2     # — amplitude spectra —————
3     amplitude_av_raw      = sum(amplitude_raw, dims=2) / size(amplitude_raw, 2)
4     amplitude_av_db_raw   = 20 .* log10.(amplitude_av_raw)
5     amplitude_av_db_raw   = amplitude_av_db_raw .- maximum(amplitude_av_db_raw)
6
7     amplitude_av_processed = sum(amplitude_processed, dims=2) /
size(amplitude_processed, 2)
8     amplitude_av_db_processed = 20 .* log10.(amplitude_av_processed)
9     amplitude_av_db_processed = amplitude_av_db_processed .-
maximum(amplitude_av_db_processed)
10
11     p7b = plot(positive_freq_raw, amplitude_av_db_raw,
12               label="Unprocessed", linewidth=3, color=:black, grid=false,
13               xlabel="Frequency (Hz)", ylabel="Amplitude (dB)",
14               fontsize=12, xtickfontsize=12, ytickfontsize=12,
15               guidefontsize=12, legendfontsize=12)
16     plot!(p7b, positive_freq_raw, amplitude_av_db_processed,
17           label="Processed", linewidth=3, color=:black, linestyle=:dash)
18     xlims!(p7b, 0, 80)
19     ylims!(p7b, minimum(amplitude_av_db_processed), 0)
20
21     # — phase variance spectra —————
22     r_raw_5000      = [1 - circ_r(phase_raw[i, 2500:7500])      for i in
1:size(phase_raw, 1)]
23     r_processed_5000 = [1 - circ_r(phase_processed[i, 2500:7500]) for i in
1:size(phase_processed, 1)]
24
25     p7c = plot(positive_freq_raw, r_raw_5000,
26               label=false, lw=3, color=:black, grid=false,
27               xlabel="Frequency (Hz)", ylabel="Circular (Phase) Variance",
28               fontsize=12, xtickfontsize=12, ytickfontsize=12,
29               guidefontsize=12, legendfontsize=12)
30     plot!(p7c, positive_freq_processed, r_processed_5000,
31           label=false, lw=3, linestyle=:dash, color=:black)
32     xlims!(p7c, 0, 80)
33     ylims!(p7c, -0.1, 1)
34
35     plot(p7b, p7c,
36          layout=(1, 2),
37          size=(1000, 400),
38          left_margin=8Plots.mm,
39          bottom_margin=8Plots.mm,
40          dpi=300)
41 end

```

These observations suggest that seismic processing should be evaluated not only by amplitude and SNR metrics but also by its ability to systematically reduce phase variance. Monitoring phase variance provides a direct means to assess whether a given processing step improves or degrades phase integrity and offers a quantitative guide for parameter selection. Although we examine only two stages here, raw data and fully time-processed data, the same framework can be applied step by step

throughout the processing sequence.

Identifying frequency bands with acceptable phase variance naturally leads to strategies for mitigating phase variability within those bands. In practice, seismic processing aims not only to diagnose phase distortions but also to compensate for them. Bakulin et al. (2023) showed that when the underlying signal is consistent across traces, the phase of the clean signal can be recovered using phase masking based on local ensembles, because the phase spectrum of the ensemble expectation matches that of the signal. In this case, the circular mean phase provides a convenient, amplitude-independent estimate of the signal phase.

DISCUSSION

Treating seismic phase as a circular variable reframes phase variability from a difficult, trace-level attribute into a measurable ensemble property that can be evaluated systematically across frequency and offset. Circular variance, here called phase variance $V(\omega)$, operates directly on wrapped phases and avoids the ambiguities of phase unwrapping. Low $V(\omega)$ corresponds to tightly clustered phases and coherent signal behavior, whereas high $V(\omega)$ indicates strong phase dispersion caused by noise, scattering, or other incoherent effects. In addition, the circular mean phase provides a useful estimate of the underlying signal phase within an ensemble. Although not the focus of this study, this estimate may support additional quality control of phase trends and serve as a key ingredient in phase-enhancement approaches such as seismic time-frequency masking (Bakulin et al., 2023) and phase-based filtering.

Synthetic tests confirm that $V(\omega)$ reliably recovers imposed phase variability as a function of frequency and offset. Application to field data shows that conventional time-domain processing primarily reduces phase variance at low frequencies and mid-to-far offsets. At higher frequencies, phase variance often remains elevated, even when amplitude spectra suggest apparent bandwidth extension. This discrepancy highlights a key limitation of amplitude-based diagnostics and motivates defining an effective bandwidth in terms of phase coherence rather than amplitude alone.

Phase variance maps and spectra provide a direct way to identify where phase information becomes reliable after processing. By selecting an application-dependent threshold on $V(\omega)$, practitioners can objectively delineate the frequency range suitable for phase-sensitive workflows, such as AVO analysis, full-waveform inversion, and migration. Unlike visual inspection or SNR metrics, $V(\omega)$ isolates phase behavior and reveals frequency-dependent limitations that would otherwise remain hidden. The statistical stability of $V(\omega)$ and its dependence on ensemble size are examined in Appendix A.

Conclusions

We have introduced circular statistics as a new and necessary framework for analyzing seismic phases. By treating phase as a circular variable rather than a trace-level artifact, phase variability becomes a measurable ensemble property that can be quantified, tracked, and acted upon systematically. Circular variance is not merely another quality-control attribute but a physically grounded descriptor of phase coherence that directly reflects the reliability of phase information across frequency and offset.

Our results show that amplitude-based diagnostics alone can substantially overestimate usable bandwidth. Conventional processing often amplifies high-frequency amplitudes while leaving phase behavior noise-dominated. Phase variance directly exposes this limitation, enabling the definition of an effective bandwidth based on phase coherence rather than spectral amplitude. This represents a fundamental shift in how seismic data quality is assessed for phase-sensitive applications.

We propose using phase variance as a standard prestack diagnostic, evaluated alongside amplitude spectra and SNR. Tracking $V(\omega)$ through a processing sequence provides an objective way to identify where phase integrity improves or deteriorates, to guide parameter selection, and to prevent the propagation of phase distortions that would otherwise remain invisible. Importantly, this diagnostic view naturally enables phase-conditioning and masking strategies driven by explicit objectives, such as achieving a prescribed level of phase fidelity for imaging or inversion, rather than by visual QC or amplitude-based criteria.

More broadly, this work establishes circular statistics as a practical and extensible foundation for seismic phase analysis and provides the statistical basis for future phase-enhancement and masking approaches that operate directly on ensemble phase behavior. The framework applies equally to QC, processing design, and phase-sensitive imaging and inversion. We anticipate that phase-based diagnostics and conditioning methods built on circular statistics will become increasingly important as seismic workflows move toward higher frequencies, denser sampling, and greater sensitivity to subtle phase effects.

Acknowledgments

We acknowledge Fairfield Geotechnologies for granting permission to use the data presented in this study.

References

Abbott, J. G., and T. H. Thurstone, 1979, Acoustic speckle: Theory and experimental analysis: *Ultrasonic Imaging*, 1, 303–324, doi: 10.1016/0161-7346(79)90021-2.

Bakulin, A., I. Silvestrov, I. Dmitriev, D. Neklyudov, M. Protasov, K. Gadylyshin, and others, 2020, Nonlinear beamforming for enhancement of 3D prestack land seismic data: *Geophysics*, 85, 1MJ–Z13, doi: 10.1190/geo2019-0631.1.

Bakulin, A., D. Neklyudov, and I. Silvestrov, 2022a, Multiplicative seismic noise caused by small-scale near-surface scattering and its transformation during stacking: *Geophysics*, 87(5), V419–V435, doi: 10.1190/geo2021-0632.1.

Bakulin, A., I. Silvestrov, and M. Protasov, 2022b, Signal-to-noise ratio computation for challenging land single-sensor seismic data: *Geophysical Prospecting*, 70(3), 629–638, doi: 10.1111/1365-2478.13197.

Bakulin, A., D. Neklyudov, and I. Silvestrov, 2023, Seismic time-frequency masking for suppression of seismic speckle noise: *Geophysics*, 88(5), V371–V385, doi: 10.1190/geo2022-0701.1.

Bakulin, A., D. Neklyudov, and I. Silvestrov, 2024, The impact of receiver arrays on suppressing seismic speckle scattering noise caused by meter-scale near-surface heterogeneity: *Geophysics*, 89, V551–V565, doi: 10.1190/geo2023-0489.1.

Bakulin, A., A. Rohatgi, and S. Fomel, 2025, Statistical analysis of seismic phase variability in dense data: 86th EAGE Annual Conference & Exhibition, EAGE Technical Program Expanded Abstracts, doi: 10.3997/2214-4609.2025101166.

Cary, P., and N. Nagarajappa, 2014a, Robust surface-consistent residual statics and phase correction — Part 1: GeoConvention 2014, FOCUS: Adapt, Refine, Sustain, Calgary, Alberta, Canada.

Cary, P., and N. Nagarajappa, 2014b, Robust surface-consistent residual statics and phase correction — Part 2: GeoConvention 2014, FOCUS, Calgary, Alberta, Canada.

Feigl, K. L., R. M. Smith, F. D. Behn, and A. F. Sheehan, 2009, A method for modeling radar interferograms without phase unwrapping: *Geophysical Journal International*, 176, 491–497, doi: 10.1111/j.1365-246X.2008.03931.x.

Fisher, N. I., 1993, *Statistical analysis of circular data*: Cambridge University Press.

Fomel, S., and van der Baan, M., 2014, Local skewness attribute as a seismic phase detector: *Interpretation*, 2, SA49–SA56, doi: 10.1190/INT-2013-0080.1.

Goodman, J. W., 2007, *Speckle phenomena in optics: Theory and applications*: Roberts & Company.

Gosselin, J. M., 2022, Probabilistic inversion of circular phase spectra: Application to two-station phase-velocity dispersion estimation: *Geophysical Journal International*, 229, 1492–1517, doi: 10.1093/gji/ggac021.

Holt, R., and Lubrano, A., 2020, Stabilizing the phase of onshore 3D seismic data: *Geophysics*, 85(6), V473–V479, doi: 10.1190/geo2019-0695.1.

Jiang, Y., J. Hu, Z. Li, and Q. Liu, 2019, Bayesian inversion of wrapped satellite interferometric phase: *Geophysical Journal International*, 219, 1500–1518, doi: 10.1093/gji/ggz379.

Mardia, K. V., and P. E. Jupp, 2000, *Directional statistics*: John Wiley & Sons.

Meunier, J., 2011, *Seismic acquisition from yesterday to tomorrow: 2011 Distinguished Instructor Course*, Distinguished Instructor Series, No. 14, SEG and EAGE.

Mohammad, H. H., S. Z. Satari, and W. N. S. W. Yusoff, 2021, Review on circular–linear regression models: *Journal of Physics: Conference Series*, 1988, 012108, doi: 10.1088/1742-6596/1988/1/012108.

Newman, P., and J. T. Mahoney, 1973, Patterns—With a pinch of salt: *Geophysical Prospecting*, 21(2), 197–219, doi: 10.1111/j.1365-2478.1973.tb00023.x.

Nugraha, A. A., K. Kobayashi, T. Toda, S. Sakti, and S. Nakamura, 2019, A deep generative model of speech complex spectrograms: *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 27, 63–76, doi: 10.1109/TASLP.2018.2879810.

Reilly, J. M., A. P. Shatilo, and Z. J. Shevchek, 2010, The case for separate sensor processing: Meeting the imaging challenge in a producing carbonate field in the Middle East: *The Leading Edge*, 29, 1240–1249, doi: 10.1190/1.3496914.

Rohatgi, A., A. Bakulin, and S. Fomel, 2024, Analyzing the impact of additive and multiplicative noise on seismic data analysis: *Fourth International Meeting for Applied Geoscience & Energy*, SEG Technical Program Expanded Abstracts, doi: 10.1190/image2024-4086176.1.

Rohatgi, A., A. Bakulin, and S. Fomel, 2025, Data-driven analysis of seismic phase using circular statistics: *The Leading Edge*, 44, 683–691, doi: 10.1190/tle44090683.1.

Sato, H., M. C. Fehler, and T. Maeda, 2012, *Seismic wave propagation and scattering in the heterogeneous Earth*, 2nd ed.: Springer, doi: 10.1007/978-3-642-23029-5.

Schimmel, M., and J. Gallart, 2007, Frequency-dependent phase coherence for noise suppression in seismic array data: *Journal of Geophysical Research: Solid Earth*, 112, B04303, doi:

10.1029/2006JBo04680.

Schimmel, M., and H. Paulssen, 1997, Noise reduction and detection of weak, coherent signals through phase-weighted stacks: *Geophysical Journal International*, 130, 497–505, doi: 10.1111/j.1365-246X.1997.tb05664.x.

Shatilo, A. P., 1992, Seismic phase unwrapping: Methods, results, problems: *Geophysical Prospecting*, 40, 211–225, doi: 10.1111/j.1365-2478.1992.tb00080.x.

Stork, C., 2020, How does the thin near surface of the Earth produce 10–100 times more noise on land seismic data than on marine data?: *First Break*, 38, 67–75, doi: 10.3997/1365-2397.fb2020062.

Takamichi, S., K. Kobayashi, T. Toda, and H. Saruwatari, 2018, Phase reconstruction from amplitude spectrograms based on a von Mises-distribution deep neural network: *Interspeech 2018 Proceedings*, 2027–2031.

Taner, M. T., F. Koehler, and K. A. Alhilali, 1974, Estimation and correction of near-surface time anomalies: *Geophysics*, 39, 441–463, doi: 10.1190/1.1440441.

Taner, M. T., and F. Koehler, 1981, Surface-consistent corrections: *Geophysics*, 46, 17–22, doi: 10.1190/1.1441133.

van der Baan, M., 2008, Time-varying wavelet estimation and deconvolution by kurtosis maximization: *Geophysics*, 73, V11–V18, doi: 10.1190/1.2831936.

van der Baan, M., and Fomel, S., 2009, Nonstationary phase estimation using regularized local kurtosis maximization: *Geophysics*, 74, A75–A80, doi: 10.1190/1.3213533.

von Mises, R., 1918, Über die "Ganzzahligkeit" der Atomgewichte und verwandte Fragen: *Physikalische Zeitschrift*, 19, 490–500.

Zebker, H. A., and Y. Lu, 1998, Phase unwrapping algorithms for radar interferometry: A systematic comparison: *Journal of the Optical Society of America A*, 15, 586–600, doi: 10.1364/JOSAA.15.000586.

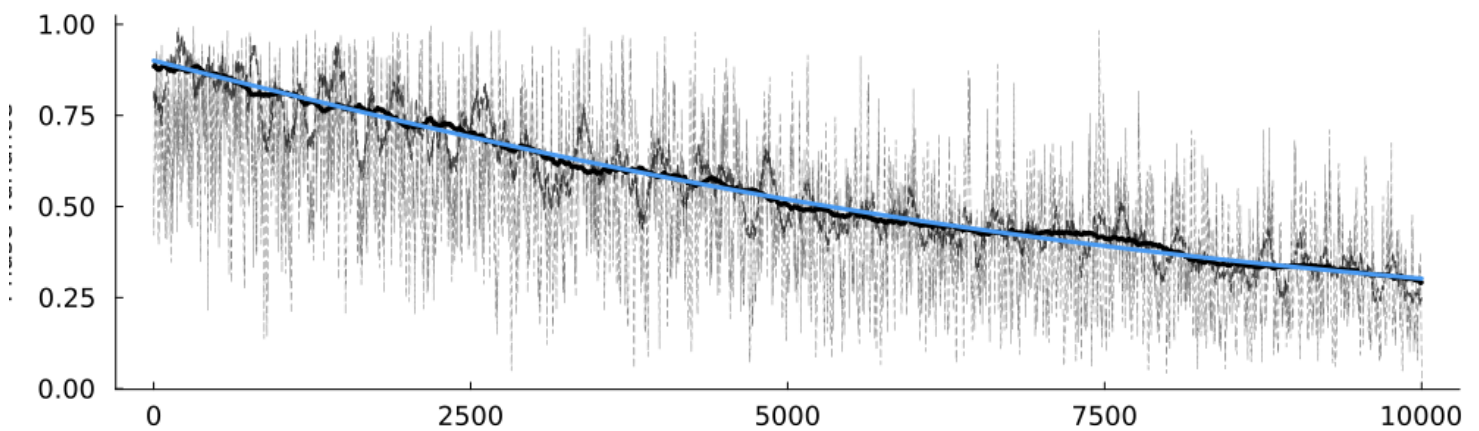
Ziolkowski, A., P. Hanssen, R. Gatliff, X.-Y. Li, and H. Jakubowicz, 2003, Use of low frequencies for sub-basalt imaging: *Geophysical Prospecting*, 51, 169–182, doi: 10.1046/j.1365-2478.2003.00363.x.

APPENDIX A: EFFECT OF ENSEMBLE SIZE ON THE STABILITY OF PHASE VARIANCE ESTIMATE

Circular variance is a statistical quantity estimated from a finite ensemble of phase samples. As such, the variance itself is a random variable whose stability depends on the number of traces included in the ensemble. Importantly, small ensemble sizes do not render phase-variance estimates uninterpretable; rather, they increase estimator variability and make interpretation more sensitive to local fluctuations. Interpretable phase-variance estimates therefore require an ensemble size appropriate to the objective of the analysis. To assess the effect of ensemble size, we computed circular variance using sliding windows containing 10, 100, and 1000 traces (Figure A1). For small ensembles, the estimated variance exhibits strong local fluctuations due to limited sampling, revealing fine-scale detail but with increased statistical scatter. As the number of traces increases, these fluctuations are progressively suppressed, and the estimated variance converges toward a smooth and stable trend that reflects the underlying phase dispersion rather than estimator noise. Importantly, the large-scale trend in phase variability is preserved across all window sizes, while increasing the ensemble size improves statistical stability.

This behavior is expected and reflects the nested nature of the problem: individual phase values are random, and circular variance is itself an estimate derived from those random samples. Consequently, the ensemble size must be chosen based on the analysis objective. When the goal is to resolve only the underlying trend in phase variability, large ensembles are required, particularly when circular variance is used as a diagnostic or quality-control attribute where stability and robustness are critical. Smaller ensembles may be useful when emphasizing localized structure, but at the expense of increased variability and interpretational uncertainty.

This distinction underpins the consistent use of different ensemble sizes throughout this study and ensures that circular variance is applied appropriately to each task.



```

1 begin
2   variance_perturbed_10 = local_phase_variance(phase_offset; batch_size=10)
3   variance_perturbed_100 = local_phase_variance(phase_offset; batch_size=100)
4   variance_perturbed_1000 = local_phase_variance(phase_offset; batch_size=1000)
5
6   variance_out_10 = variance_perturbed_10[6, :]
7   variance_out_100 = variance_perturbed_100[6, :]
8   variance_out_1000 = variance_perturbed_1000[6, :]
9
10  p_appendix = plot(variance_out_10, label="N = 10", lw=0.5,
11                    linestyle=:dash, color=:black, alpha=0.5,
12                    ylabel="Phase Variance", xlabel="Trace Index",
13                    grid=false, fontsize=12, labelfontsize=12,
14                    legendfontsize=12, tickfontsize=12,
15                    xguidefontsize=12, yguidefontsize=12,
16                    size=(1000, 300))
17
18  plot!(p_appendix, variance_out_100, label="N = 100",
19        lw=1, linestyle=:dash, color=:black, alpha=0.7)
20
21  plot!(p_appendix, variance_out_1000, label="N = 1000",
22        lw=3, color=:black, grid=false)
23
24  plot!(p_appendix, var_imposed, label="Imposed",
25        lw=3, color="#499af2", grid=false)
26
27  p_appendix
28 end

```